



哈爾濱工業大學(深圳)

HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

Kairix

设计文档

参赛队名 Unicus

队伍成员 颜晨、雷鑫言、萧鹏

指导老师 夏文、仇洁婷

2026 年 6 月 30 日

摘要

Kairix是一个使用Rust实现、支持RISC-V和LoongArch指令集架构的宏内核操作系统,基于有栈协程开发,具备全面的功能以及开源属性。

在开发过程中, Kairix充分借鉴了往年PhoenixOS、Nighthawk、TitanixOS、Chronix等优秀竞赛内核的设计经验, 参考了Linux等工业级操作系统的实现思路。我们总结、提炼这些优秀工作的精华, 团队成员分工协作, 共同开发出高效、稳定、模块化、可扩展的操作系统内核Kairix。

我们队伍在内核中实现了诸多方面的优化与改良, 包括模块化的RISC-V/LoongArch多架构适配、多文件系统的支持、自研的网络栈等。

Kairix内核在功能完整性和系统能力方面进行了多项扩展与完善: **支持多核处理器运行**, 能够完成多处理器环境下的任务调度与并发执行提高执行效率; **同时兼容双架构**, 提升了内核在不同硬件平台上的适配能力; **实现并完善了自研网络栈**, 为上层应用提供基础网络通信支持; 此外, **引入页缓存刷写机制**, 使文件数据能够从页缓存可靠同步到底层存储设备, 增强了文件系统运行的稳定性和数据一致性。

截至6月29日17时13分, Kairix在初赛阶段的得分情况如下图所示, 在参赛队伍中排名前列:

#	用户名	队伍	学校	提交次数 (ASC)	最后提交时间 (ASC)	basic-glibc-la	basic-glibc-rv	basic-musl-la	basic-musl-rv	busybox-glibc-la	busybox-glibc-rv	busybox-musl-la	busybox-musl-rv
1	T202610574999863	夺金入场	华南师范大学	36	2026-06-29 14:59:16	91.0	91.0	91.0	91.0	57.0	57.0	57.0	57.0
2	T202610486999845	alREDy	武汉大学	262	2026-06-28 16:24:33	100.0	102.0	102.0	102.0	54.0	54.0	54.0	54.0
3	T202610459999616	PulseOS	郑州大学	164	2026-06-29 11:25:21	102.0	102.0	102.0	102.0	54.0	54.0	54.0	54.0
4	T202610617999569	每天睡满八小时	重庆邮电大学	200	2026-06-24 02:43:21	102.0	102.0	102.0	102.0	54.0	54.0	54.0	54.0
5	T2026181239910993	Unicus	哈尔滨工业大学 (深圳)	53	2026-06-28 01:14:05	102.0	102.0	102.0	102.0	54.0	54.0	54.0	54.0

图 0-1: 6-30 排行榜情况

Kairix各个模块完成情况如下表：

模块	完成情况
进程管理	有栈协程、多核调度。进程线程分离设计。
内存管理	内核空间动态映射、缺页异常驱动的懒分配、写时复制、支持零页分配、用户指针检查、动态链接、共享内存映射
文件系统	实现了类Linux的虚拟文件系统，其中功能包括路径解析缓存、页缓存加速、脏页回刷、延迟写、支持挂载、支持Ext4/Ext3/Ext2/Fat32磁盘文件系统，内存文件系统、进程文件系统、设备文件系统。
信号机制	支持标准信号与实时信号的相关接口、支持用户自定义信号处理、线程信号掩码、同步异常信号投递。
设备驱动	支持硬件中断、MMIO驱动、PCI驱动和部分串口驱动、解析部分设备树信息。
网络模块	自研网络栈、支持TCP UDP套接字、支持本地回环设备和IPv4协议栈。
架构管理	基于开源多架构硬件抽象层polyhal、支持RISC-V、LoongArch双架构。

表 0-1: 模块完成情况

摘 要	2
1 概述	8
1.1 Kairix整体架构	8
1.2 Kairix文件架构	9
2 进程设计与管理	11
2.1 多核有栈协程设计	11
2.1.1 为什么不选择无栈协程设计	11
2.1.2 有栈协程设计	11
2.2 进程线程分离设计	16
2.2.1 资源分配层面：进程线程分离	16
2.2.2 调度层面：线程作为调度单位	18
2.2.3 任务的状态	21
2.3 任务调度与任务执行	23
3 内存管理	24
3.1 物理内存管理	24
3.2 内核动态内存分配	25
3.3 虚拟地址空间	25
3.3.1 虚拟地址空间布局	25
3.3.2 共享内核地址空间	27
3.4 用户地址空间管理	27
3.4.1 懒分配	28
3.4.2 写时复制	29
4 文件系统设计	30
4.1 虚拟文件系统	30
4.2 核心数据结构设计	31
4.2.1 FsType	31
4.2.2 SuperBlock	32
4.2.3 Dentry	33
4.2.4 Dcache	34
4.2.5 Inode	34
4.2.6 File	35
4.2.7 Fd Table	36

4.3 磁盘文件系统实现	36
4.3.1 FAT32 文件系统	36
4.3.2 ext4 文件系统	36
4.4 非磁盘文件系统	37
4.4.1 procfs	37
4.4.2 devfs	38
4.4.3 tmpfs	39
4.5 页缓存	39
4.5.1 Page	39
4.5.2 PageCache	40
4.6 其他数据结构	40
4.6.1 pipe	40
5 进程间通信	41
5.1 信号机制	41
5.1.1 数据结构设计	41
5.1.2 信号处理函数	42
5.1.3 信号上下文管理	44
5.1.4 信号的处理整体流程	45
5.2 管道机制	46
5.3 共享内存机制	47
5.3.1 共享内存数据结构	47
5.3.2 共享内存系统调用	48
6 系统调用	49
6.1 系统调用接口	49
6.2 用户态与内核态切换	49
6.2.1 陷阱处理机制	50
6.2.2 内核态到用户态	50
6.3 系统调用处理	51
6.4 系统调用分发与错误处理	51
6.5 系统调用列表	51
7 网络模块	53
7.1 概述	53

7.2	数据包缓冲区	54
7.3	设备层	54
7.4	链路层	55
7.5	网络层	55
7.6	传输层	56
7.6.1	UDP	56
7.6.2	TCP	58
7.7	套接字层与文件系统集成	59
7.8	异步网络 I/O 模型	60
8	设备驱动	61
8.1	设备解析	61
8.1.1	RISC_V MMIO 设备解析	61
8.1.2	LoongArch PCI 设备解析	61
8.2	块设备驱动	62
8.3	字符设备驱动	62
8.4	网络设备驱动	63
9	多架构设计	64
9.1	RISC-V和LoongArch指令集架构	64
9.1.1	RISC-V	64
9.1.2	LoongArch	64
9.2	Kairix硬件抽象层实现	64
9.2.1	虚拟地址抽象	65
9.2.2	异常处理	66
10	AI工具的使用	69
10.1	使用场景	69
10.2	AI生成内容范围	71
10.3	AI工具的幻觉	71
10.4	反思	72
11	总结与展望	74
11.1	主要工作成果	74
11.2	开发过程	75
11.3	开发经验总结	75

11.4 遇到的问题和解决方案	76
11.4.1 网络回环设备收发问题	76
11.4.2 旧设计产生的效率问题	76
11.4.3 LoongArch架构的内存错误问题	77
11.5 项目意义	78
11.6 未来发展计划	78
12 参考往届内核	79

1 概述

Kairix是一个支持多核与多架构的宏内核操作系统内核。项目以rCore为基础进行开发，并在设计与实现过程中参考了多个优秀内核的成熟经验，吸收其在内存管理、设备驱动、多架构抽象等方面的设计优点，在此基础上进行整合、改进与扩展。

与此同时，Kairix并未局限于既有实现路径，而是在若干关键模块上进行了自主探索与创新。在线程与调度机制方面，Kairix**重新采用有栈协程模型**，而非沿用无栈协程方案，以获得更贴近传统内核执行流的上下文管理能力；在网络子系统方面，Kairix**自研网络协议栈**，逐步支持TCP/IP相关功能，力图补足以往同类教学内核中网络能力不足的问题；在文件系统缓存方面，Kairix实现了统一页缓存机制，并支持**脏页回刷**，为文件系统性能优化和数据一致性提供了基础。

我们希望Kairix不仅是一个完成竞赛目标的内核项目，也能成为社区中可参考、可复用、可继续演进的实现样例。

1.1 Kairix整体架构

Kairix整体结构为宏内核，自底向上分为硬件平台、启动层、硬件抽象层、内核核心层和用户空间。如图所示。在**Machine层面**，内核运行于QEMU virt硬件平台之上，RISC-V64通过OpenSBI启动，LoongArch64则由QEMU直接加载内核镜像启动。其上由polyhal硬件抽象层负责屏蔽双架构差异，统一提供页表、Trap、上下文切换、时钟和中断等基础能力。在**内核模式层面**，Kairix主要划分为任务调度、内存管理、VFS与存储、设备驱动和网络栈五个模块。其中任务调度方面，采用PCB/TCB分离设计、有栈协程、多核调度；内存管理部分，支持懒分配、写时复制、动态链接、共享内存映射；文件系统部分，支持多文件系统的挂载，同时支持页缓存和脏页的刷回；设备驱动方面，抽象出统一接口，支持基于MMIO、PCI/ECAM和FDT的设备发现与访问；在网络部分，自研IPv4网络栈，支持Ethernet、ARP、ICMP、UDP、TCP、RAW socket、路由和本地回环等功能。在**用户模式层面**，用户程序通过POSIX风格系统调用接口进入内核态。

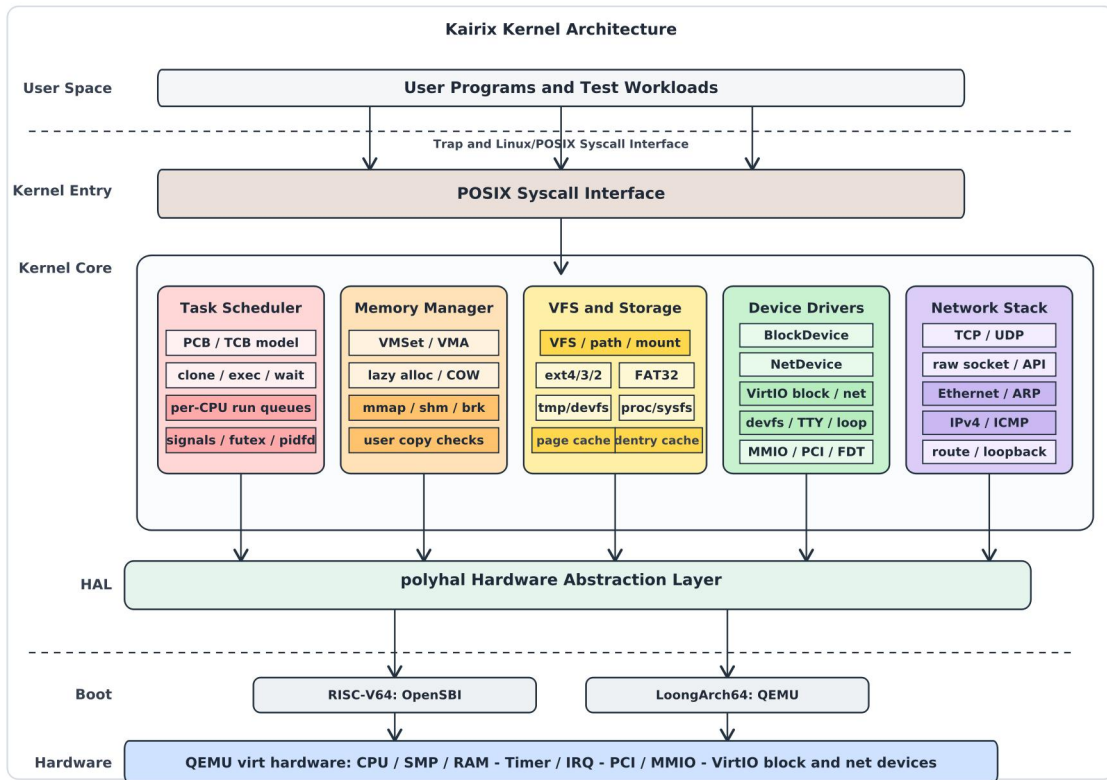


图 1-1: 总体架构

1.2 Kairix文件架构

```

1 Kairix/
2 |— os/                                # 内核主体代码：进程、内存、VFS、网络、驱动、系统调用等
3 |   |— src/
4 |   |   |— main.rs                    # 内核入口，完成初始化后进入任务调度
5 |   |   |— config.rs
6 |   |   |— logging.rs                # 日志初始化
7 |   |   |— console.rs               # 内核控制台输出封装
8 |   |   |— error.rs                 # 内核错误码与系统调用错误类型
9 |   |   |— lang_items.rs            # no_std panic/语言项支持
10 |   |   |— timer.rs                # 时钟与定时器相关逻辑
11 |   |   |— embedded.rs             # 启动时内置文件安装
12 |   |   |— sbi.rs                  # RISC-V SBI调用封装
13 |   |   |— sbi_la.rs               # LoongArch 固件调用封装
14 |   |   |— entry.asm               # 早期汇编入口
15 |   |   |— link_app.S              # 用户程序链接辅助
16 |   |   |— linker-*.ld             # RISC-V/LoongArch/QEMU链接脚本
17 |   |   |— arch/                   # 架构相关代码，包含riscv_dir与loongarch_dir
18 |   |   |— boards/                 # 板级配置，当前主要面向 QEMU virt
19 |   |   |— devices/               # 通用设备抽象
20 |   |   |— drivers/               # 设备驱动，包含VirtIO块设备与PCI探测
21 |   |   |— fs/                    # 文件系统子系统
22 |   |   |— mm/                    # 内存管理
23 |   |   |— net/                   # 网络协议栈
24 |   |   |— socket/                # socket层
25 |   |   |— sync/                  # 同步原语
26 |   |   |— syscall/               # 系统调用实现

```

27				task/	# 进程/线程管理、调度器、PID、上下文切换
28				trap/	# trap/异常/中断处理入口与上下文
29				.cargo/	# 内核crate本地Cargo配置
30				vendor/	# 内核构建使用的离线依赖
31				user/	# 用户态运行时库与测试/示例程序
32				src/	
33				lib.rs	# 用户态运行时入口、堆初始化、系统调用安全封装
34				syscall.rs	# 用户态系统调用号与ecall/syscall汇编封装
35				console.rs	# 用户态print/println与输入输出辅助
36				lang_items.rs	# 用户态no_std panic/语言项支持
37				linker.ld	# 用户程序链接脚本
38				bin/	# 用户程序与测试入口
39				.cargo/	# 用户程序crate本地Cargo配置
40				vendor/	# 用户程序构建使用的离线依赖
41				polyhal/	# 多架构硬件抽象层
42				polyhal/	# HAL核心实现
43				polyhal-boot/	# 启动入口与架构初始化
44				polyhal-trap/	# trap/中断上下文抽象
45				polyhal-macro/	# 架构相关过程宏
46				example/	# HAL示例程序
47				bootloader/	# 启动固件，当前包含rustsbi-qemu.bin
48				lwext4_rust/	# ext4文件系统绑定与lwext4 C库
49				rust-fatfs/	# FAT/FAT32文件系统实现
50				iperf/	# iperf网络性能测试工具源码
51				netperf-2.7.0/	# netperf网络性能测试工具源码
52				tools/	# 镜像与文件系统工具
53				patches/	# 兼容补丁与移植补丁
54				docs/	# 项目文档、架构图与测试说明
55				.devcontainer/	
56				.vscode/	
57				Makefile	
58				rust-toolchain.toml	
59				Unicus初赛文档.pdf	
60				Unicus初赛PPT.pdf	
61				README.md	
62				LICENSE	
63				.gitignore	
64				.dockerignore	
65				dev-env-info.md	

2 进程设计与管理

2.1 多核有栈协程设计

2.1.1 为什么不选择无栈协程设计

在调研我们学校往届的内核时，我们发现往届队伍大多采用无栈协程设计，利用状态机来进行切换上下文，这样的优势是无需分配内核栈，节省大量内存空间，拥有着更低的调度成本和更低的内存占用。缺点是在较深的嵌套调用的时候，很难返回最初的调用位置，并且代码会有async层层往上污染，且代码调试极为困难。而我们团队设计内核的初衷是：希望Kairix能够实现大量的复杂功能且同时方便后续的人在借鉴我们的内核的时候方便调试和参考，这些都不在无栈协程的优势面上。因此我们放弃了往届的无栈协程的轮子，选择重新从有栈协程出发设计进程调度。

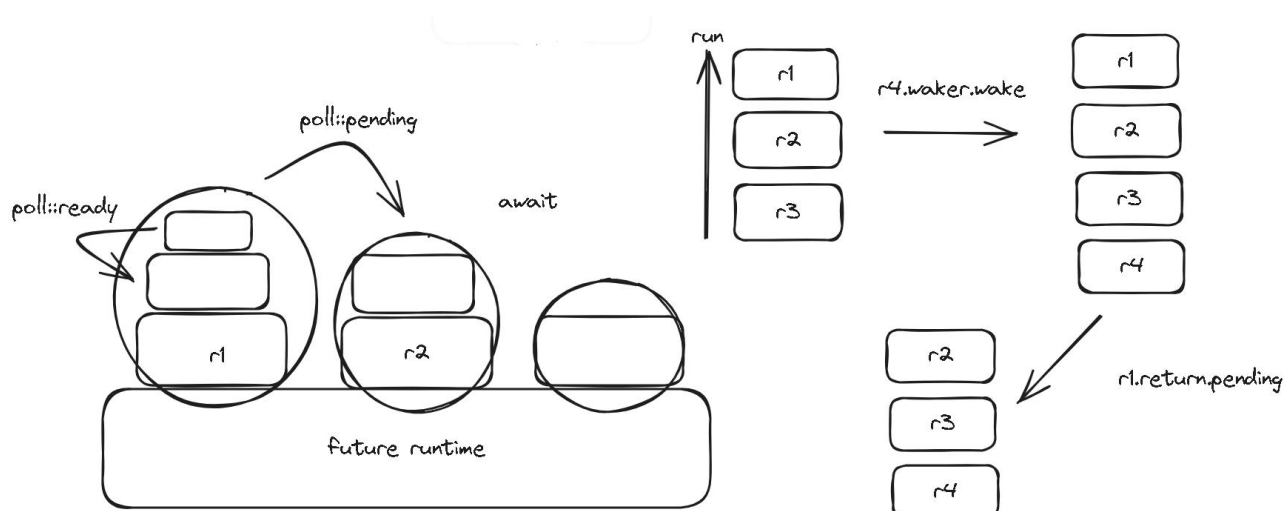


图3-1: NighthawkOS的switch-no-stack

2.1.2 有栈协程设计

在传统的有栈协程设计中，每个用户线程都有自己的内核栈，线程的上下文切换需要保存和恢复栈指针。

以 rCore 为例，如下图所示，上下文切换需要首先将当前线程的CPU寄存器保存到当前线程的内核栈上，并根据目标线程的内核栈上保存的内容来恢复寄存器，最后切换栈指针。

这种保存上下文的调度方式，导致有栈协程可以轻松的实现嵌套调用，适合多种复杂任务的处理。

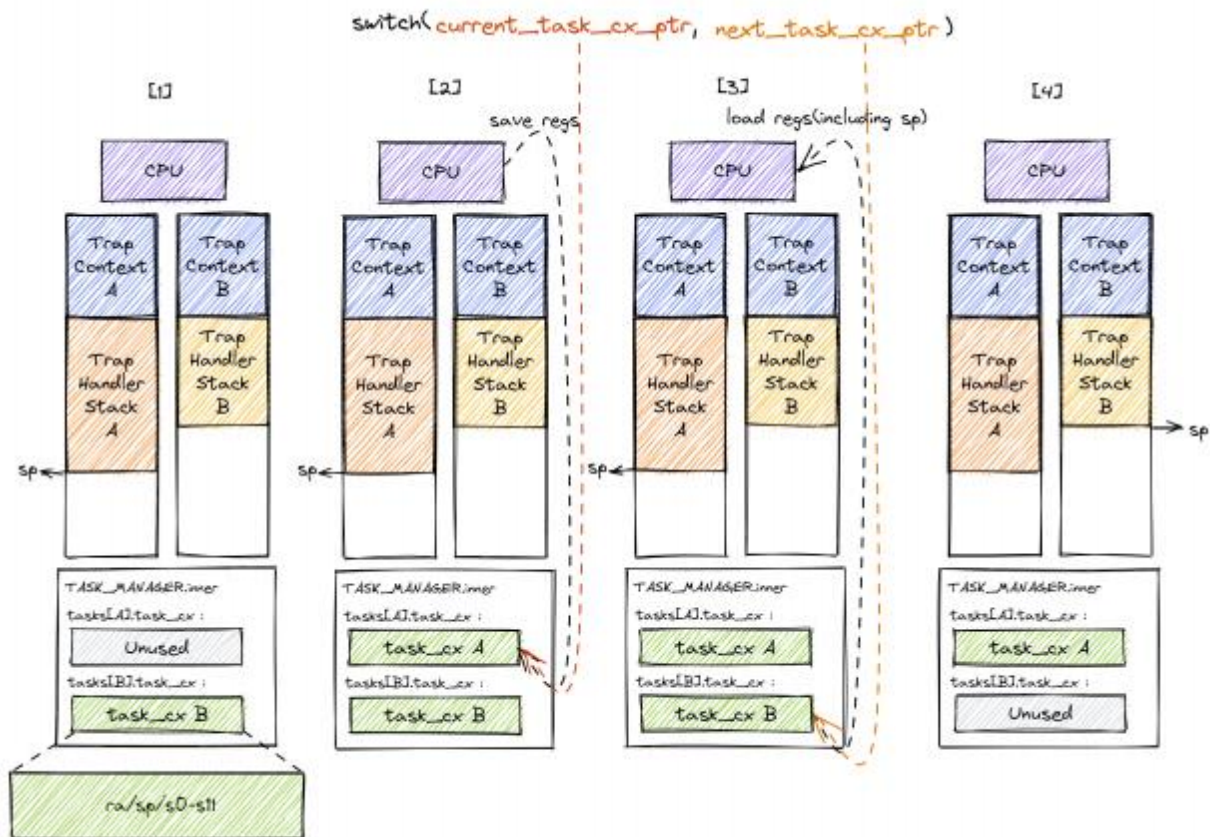


图3-2:Rcore的上下文切换

2.1.2.1 代码实现

Kairix调度线程级任务，调度核心数据结构包括：作为调度队列管理器的TaskManager，作为CPU抽象的Processor，以及作为调度基本单位的TaskControlBlock。此外，KContext负责保存内核上下文，TrapFrame负责保存用户态陷入上下文。

2.1.2.1.1 任务调度队列

Kairix支持多核运行，为每个CPU维护一个独立的TaskManager，构成per-CPU ready queue的设计方案。

每个TaskManager里面维护着多个优先级队列：

```
1 pub struct TaskManager {
2     ready_queues: [VecDeque<Arc<TaskControlBlock>>; MAX_SCHED_PRIORITY + 1],
3     high_priority_runs: usize,
4 }
```

为了确保任务调度之间的公平性和优先级，Kairix采用的方案是

1. 同一优先级队列中采用FIFO调度

2. 不同优先级之间默认从高优先级到低优先级扫描。

3. 不同优先级之间，当连续调度非零优先级任务达到HIGH_PRIORITY_BUDGET次后，调度器会优先尝试从最低优先级队列ready_queues[0]中取出一个任务运行；若该队列为空，则重新从高到低扫描所有优先级队列。这种简单的调度策略就能够保证一定的公平性和正确性。

```

1 fn fetch(&mut self) -> Option<Arc<TaskControlBlock>> {
2     if self.high_priority_runs >= HIGH_PRIORITY_BUDGET {
3         if let Some(task) = self.ready_queues[0].pop_front() {
4             self.high_priority_runs = 0;
5             return Some(task);
6         }
7         self.high_priority_runs = 0;
8     }
9     for priority in (0..=MAX_SCHED_PRIORITY).rev() {
10        if let Some(task) = self.ready_queues[priority].pop_front() {
11            if priority > 0 {
12                self.high_priority_runs += 1;
13            } else {
14                self.high_priority_runs = 0;
15            }
16            return Some(task);
17        }
18    }
19    None
20 }

```

同时，Kairix采取轻量级任务窃取机制，确保不同CPU之间具有简单负载均衡能力。当前CPU的本地任务队列为空时，调度器会尝试从其他CPU的TaskManager中获取任务。

```

1 fn fetch_task_from_cpu(cpu: usize) -> Option<Arc<TaskControlBlock>> {
2     let task = {
3         let mut manager = TASK_MANAGER[cpu].lock();
4         manager.fetch()
5     };
6     if let Some(task) = task {
7         task.clear_ready_queued();
8         Some(task)
9     } else {
10        None
11    }
12 }

```

Kairix Stackful Scheduler

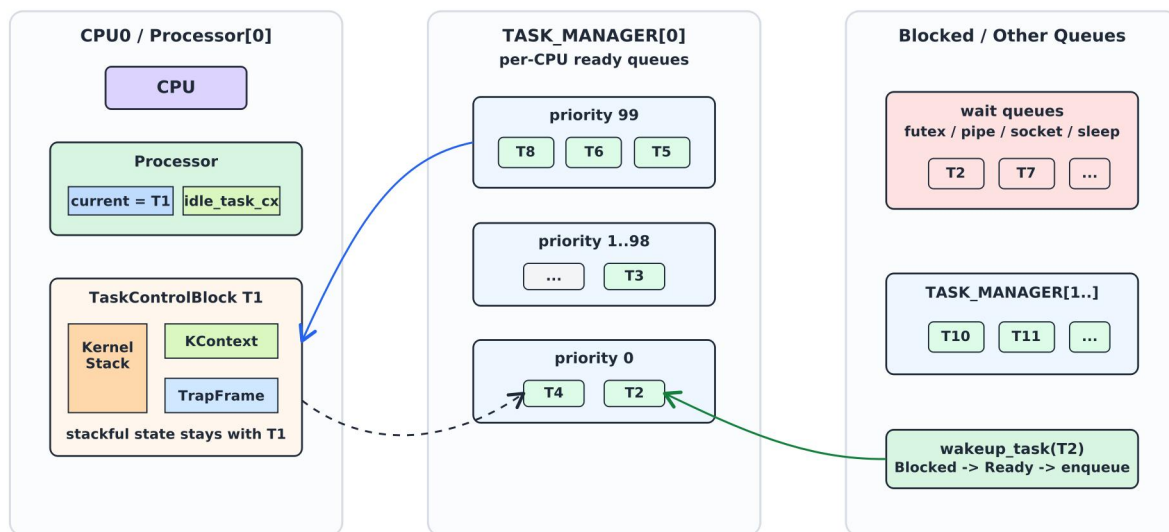


图3-3. 调度策略总览

2.1.2.1.2 上下文切换

每一个任务都有独立的上下文，从调度队列中选出下一个任务后，进行上下文的切换的时候，Kairix需要切换KContext、维护Processor。

Processor是Kairix对每个CPU调度状态的抽象：

```
1 pub struct Processor {
2     current: Option<Arc<TaskControlBlock>>,
3     idle_task_cx: KContext,
4 }
```

其中，current表示当前CPU正在运行的任务，idle_task_cx表示当前CPU的调度器上下文。

```
1 #[derive(Debug)]
2 #[repr(C)]
3 pub struct KContext {
4     /// Kernel Stack Pointer
5     ksp: usize,
6     /// Kernel Thread Pointer
7     ktp: usize,
8     /// Kernel S regs, s0 - s11, just callee-saved registers
9     /// just used in the context_switch function.
10    _sregs: [usize; 12],
11    /// Kernel Program Counter, Will return to this address.
12    kpc: usize,
13 }
```

RISC-V的KContext主要保存内核栈指针ksp、内核返回地址kpc以及s0-s11这些callee-saved registers。由于caller-saved registers按照ABI约定由调用者保存，因此上下文切换时不需要保存所有通用寄存器。

```

1  /// Kernel Context is used to switch context between kernel task.
2  #[derive(Debug)]
3  #[repr(C)]
4  pub struct KContext {
5      /// Kernel Stack Pointer
6      ksp: usize,
7      /// Kernel Thread Pointer
8      ktp: usize,
9      /// Kernel Static registers, r22 - r31 (r22 is s9, s0 - s8)
10     _sregs: [usize; 10],
11     /// Kernel Program Counter, Will return to this address.
12     kpc: usize,
13     #[cfg(feature = "fp_simd")]
14     /// Floating Point Status
15     fp_status: FpStatus,
16 }

```

同理，可以看到LoongArch的上下文保存的具体的寄存器有所不同，但是思想是一样的，即上下文切换的适合不需要保存所有的通用寄存器

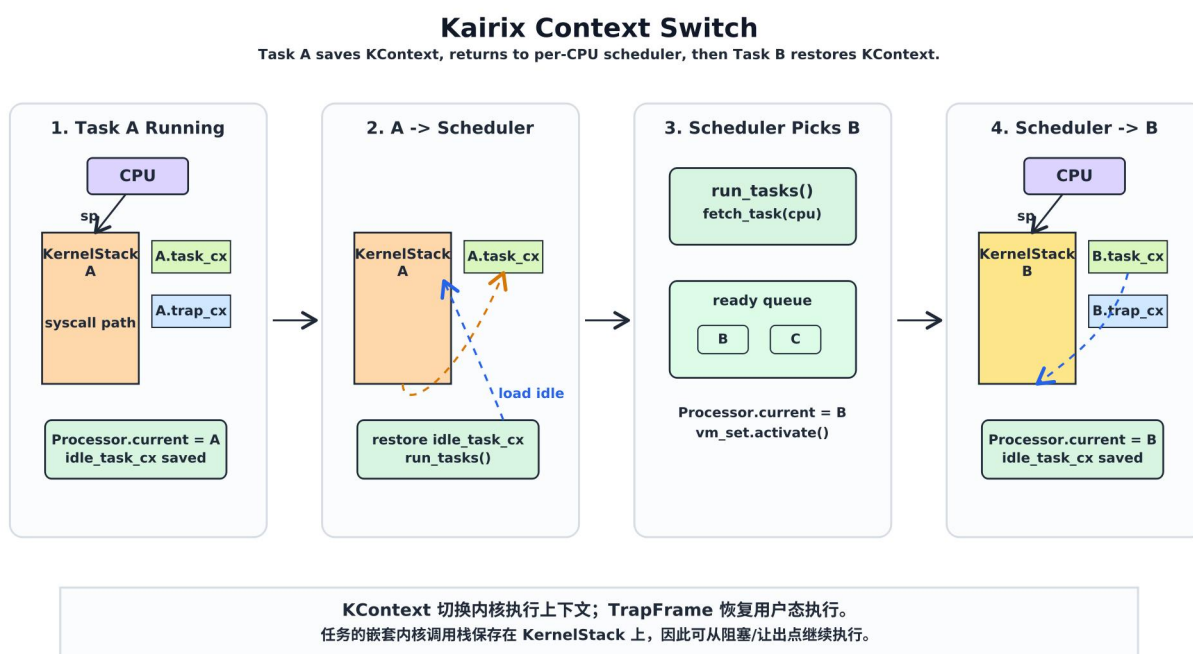


图3-4: Kairix上下文切换示意图

2.2 进程线程分离设计

进程是操作系统中资源分配的基本单位。每个进程都有自己独立的地址空间和资源，如内存、文件描述符等。线程是操作系统中CPU调度的基本单位。线程共享所在进程的地址空间和资源，但有独立的执行上下文。

在调研其他队伍的设计的时候，我们发现，部分采用无栈协程设计的内核会倾向于将可执行实体统一抽象为Task/Future，以减少调度层的重复抽象，并且在调研linux的设计的时候，发现linux并没有严格地区分进程和线程，而是通过一组统一的API来操作任务，例sys_clone系统调用通过不同的flags组合创建共享不同资源的新任务，因此进程和线程的创建本质上是类似的，这种表示方法的优势是减少重复工作量。与此同时，我们也发现很多队都将进程和线程分开设计，分别使用Process和Thread结构体表示，这种表示方式的优点是职责清晰。

综上考虑后，我们还是选择PCB/TCB分离设计的方案，因为这种设计可以让职责分开，更符合传统的设计。

2.2.1 资源分配层面：进程线程分离

Kairix在资源管理层面采用进程和线程分离的设计。进程由ProcessControlBlock表示，线程级任务由TaskControlBlock表示。

```

1 pub struct ProcessControlBlock {
2     // immutable
3     pub pid: PidHandle,
4     // mutable
5     inner: SpinNoIrqlLock<ProcessControlBlockInner>,
6 }
7 pub struct ProcessControlBlockInner {
8     pub is_zombie: bool,
9     pub is_stopped: bool,
10    pub was_continued: bool,
11    pub zombie_flag: AtomicBool,
12    pub pgid: PgidHandle,
13    /// 真实 UID
14    pub uid: u32,
15    /// 有效 UID
16    pub euid: u32,
17    /// 保存的 set-user-ID
18    pub suid: u32,
19    /// 真实 GID
20    pub gid: u32,
21    /// 有效 GID
22    pub egid: u32,
23    /// 保存的 set-group-ID

```



```

24     pub sgid: u32,
25     pub vm_set: UserVMSet,
26     pub parent: Option<Weak<ProcessControlBlock>>,
27     pub children: Vec<Arc<ProcessControlBlock>>,
28     pub exit_code: i32,
29     pub term_status: TermStatus,
30     pub fd_table: Vec<Option<Arc<dyn File + Send + Sync>>>,
31     pub fd_flags: Vec<u32>,
32     pub tasks: Vec<Option<Arc<TaskControlBlock>>>>,
33     pub task_res_allocator: RecycleAllocator,
34     pub cwd: Arc<dyn Dentry>,
35     pub time: Tms,
36     pub ustart: usize,
37     pub kstart: usize,
38     pub state: ProcessStatus,
39
40     pub pending_signals: SignalSet,
41     pub blocked_signals: SignalSet,
42     pub signals_handler: SignalHandlers,
43     pub need_signal_handle: bool,
44     pub itimer_real_deadline: Option<usize>,
45     pub itimer_real_interval: Option<usize>,
46     pub wait_waker: Option<core::task::Waker>,
47     /// 信号处理上下文栈（保存在 PCB 中，单线程场景下安全）
48     pub sig_context_stack: Vec<(TrapFrame, SignalSet)>,
49     /// ITIMER_REAL 的到期时间（微秒），None 表示未设置
50     pub alarm_deadline_us: Option<u128>,
51     /// ITIMER_REAL 的间隔时间（微秒），None 表示单次定时器
52     pub alarm_interval_us: Option<u128>,
53     /// 资源限制：文件大小上限
54     pub rlimit_fsize: Rlimit64,
55     /// 资源限制：单文件描述符最大数量
56     pub rlimit_nofile: Rlimit64,
57     /// 文件创建权限掩码
58     pub umask: u32,
59     /// prctl(PR_SET_NO_NEW_PRIVS) state.
60     pub no_new_privs: bool,
61     /// Minimal capability tracking used by Landlock tests.
62     pub has_cap_sys_admin: bool,
63     /// Landlock security domain.
64     pub landlock: LandlockDomain,
65     /// 还活着的线程数量（用于 waitpid 判断是否可以回收进程）
66     pub alive_thread_count: usize,
67     /// CLONE_VFORK 时记录需要唤醒的父任务
68     pub vfork_parent: Option<Arc<TaskControlBlock>>,
69     /// 网络命名空间 ID（0 表示初始命名空间）
70     pub net_ns_id: usize,
71     /// 子进程退出时发送给父进程的信号（clone/clone3 的 exit_signal）
72     pub exit_signal: i32,
73     /// 最近一次投递信号时携带的 siginfo（用于 pidfd_send_signal 等）
74     pub last_siginfo: Option<crate::task::signal::SigInfo>,
75 }

```

可以看到，PCB中保存的是进程级资源：地址空间、文件描述符表、当前工作目录、父子进程关系、线程列表、进程级信号处理、资源限制等。

```
1 pub struct TaskControlBlock {
2     // immutable
3     pub process: Weak<ProcessControlBlock>,
4     pub kstack: KernelStack,
5     // mutable
6     inner: SpinNoIrqLock<TaskControlBlockInner>,
7     sched_policy: AtomicU32,
8     sched_priority: AtomicI32,
9     on_cpu: AtomicUsize,
10    ready_queued: AtomicUsize,
11 }
```

2.2.2 调度层面：线程作为调度单位

虽然Kairix在资源管理层面区分PCB和TCB，但在调度层面，调度器只面对TCB。

```
1 pub struct TaskControlBlock {
2     // immutable
3     pub process: Weak<ProcessControlBlock>,
4     pub kstack: KernelStack,
5     // mutable
6     inner: SpinNoIrqLock<TaskControlBlockInner>,
7     sched_policy: AtomicU32,
8     sched_priority: AtomicI32,
9     on_cpu: AtomicUsize,
10    ready_queued: AtomicUsize,
11 }
12 pub struct TaskControlBlockInner {
13     pub res: Option<TaskUserRes>,
14     pub global_tid: usize,
15     pub trap_cx: TrapFrame,
16     pub task_cx: KContext,
17     ///
18     pub task_status: TaskStatus,
19     pub exit_code: Option<i32>,
20     ///线程退出时需要清零的用户态虚拟地址
21     pub clear_child_tid: usize,
22     /// 信号处理时保存的原始 TrapFrame
23     pub saved_sigtrapframe: Option<TrapFrame>,
24     /// 标记该线程是否被信号中断唤醒（用于阻塞系统调用返回 EINTR）
25     pub interrupted_by_signal: bool,
26     /// 线程级待处理信号（用于 tkill/tgkill 等线程定向信号）
27     pub pending_signals: crate::task::signal::SignalSet,
28     /// 线程级信号阻塞掩码
29     pub blocked_signals: crate::task::signal::SignalSet,
30     /// 是否需要处理信号
31     pub need_signal_handle: bool,
32     /// 信号处理上下文栈（用于线程自定义 handler 返回）
33     pub sig_context_stack: Vec<(TrapFrame, crate::task::signal::SignalSet)>,
34     /// sigsuspend 保存的旧信号掩码，sigreturn 后恢复
35     pub sigsuspend_old_mask: Option<crate::task::signal::SignalSet>,
```

```

36    /// 标记该线程是否已被 futex_wake 唤醒（防止丢失唤醒）
37    pub futex_woken: bool,
38    /// 标记该线程是否有待处理的唤醒（解决 lost wakeup race）
39    pub pending_wakeup: bool,
40    /// robust_list_head 指针（set_robust_list 设置）
41    pub robust_list_head: usize,
42    /// robust_list 长度（通常为 24 字节）
43    pub robust_list_len: usize,
44    /// 标记所属进程是否已被 SIGKILL 等标记为 zombie（避免 block 时竞态）
45    pub zombie_flag: AtomicBool,
46 }

```

下面介绍一下TCB的具体设计：

首先TCB大致可以分成不变和可变部分：

不可变部分：不可变部分在任务创建后基本不会改变，主要用于标识该线程所属的进程和它独立拥有的内核栈，由于这些字段不会改变，多线程环境下访问这些字段时不需要加锁，可以提高访问效率和安全性。

字段名	含义
process	指向所属ProcessControlBlock的弱引用，用于访问进程级资源
kstack	当前线程独立的内核栈，保存系统调用、异常、中断处理中的内核调用栈

可变部分：可变部分保存调度、上下文、信号、futex等线程级状态。这些字段在任务的生命周期之间可能发生变化，因此需要加锁进行保护。

字段名	含义
inner	线程内部可变状态，由SpinNoIrqLock保护
sched_policy	调度策略，目前用于保存任务调度策略字段
sched_priority	调度优先级，决定任务进入哪个优先级ready queue
on_cpu	标记任务是否已经在某个CPU上运行，防止同一任务被多个CPU同时调度

其中inner可以细分为:

字段名	含义
res	线程的用户态资源, 包括tid、用户栈等资源信息
global_tid	全局线程ID
trap_cx	用户态陷入上下文, 保存用户态寄存器现场
task_cx	内核调度上下文, 保存内核栈指针、返回地址和callee-saved registers
task_status	当前任务状态, 如Ready、Running、Blocked、Sleep、Zombie
exit_code	当前线程退出码
clear_child_tid	线程退出时需要清零的用户态地址
saved_sigtrapframe	信号处理时保存的原始TrapFrame
interrupted_by_signal	标记线程是否被信号中断唤醒, 用于阻塞系统调用返回EINTR
pending_signals	线程级待处理信号, 用于tkill、tgkill等线程定向信号
blocked_signals	当前线程的信号阻塞掩码
need_signal_handle	标记当前线程是否需要信号处理
sig_context_stack	信号处理上下文栈, 用于嵌套信号和handler返回
sigsuspend_old_mask	sigsuspend保存的旧信号掩码
futex_woken	标记线程是否已经被futex_wake唤醒
pending_wakeup	标记线程是否有待处理的唤醒
robust_list_head	robust futex链表头指针

robust_list_len	robust list的长度
zombie_flag	标记所属进程是否已经被杀死或进入zombie状态，避免线程继续阻塞

对于TCB的可变部分，从上面也能够看出，我们队伍是做了优化，调研其他队伍的时候，我们发现，大多数队伍采取的方案是直接将全部的可变字段都放入inner中，并在外面加入一个自旋锁，这样做法的优势是在并发的时候安全性得到了保证。吸取这些队伍的优势，我们队伍选择将调度频繁访问的字段使用原子变量单独保存，避免所有调度操作都需要获取大锁。

```

1 sched_policy: AtomicU32,
2 sched_priority: AtomicI32,
3 on_cpu: AtomicUsize,
4 ready_queued: AtomicUsize,
```

同时将其他可变字段放入inner中，并使用一个大锁包裹起来。这样的设计，既能够保证一定的效能，又能够确保并发时候的安全性。通过这些机制，操作系统能够更加高效地管理任务和资源。

2.2.3 任务的状态

在调研其他操作系统内核时，我们发现部分内核只是简单区分了Running和Zombie状态，并不支持当SIGSTOP信号到来时进程暂停执行。此外，部分内核对于任务在阻塞的过程中能否被信号打断，也支持得并不是很好。Linux手册提到，部分系统调用在阻塞等待的过程中可以被信号打断停止执行，返回EINTR错误；如果打断系统调用的信号的flag标志中含有SA_RESTART，又可以重启系统调用。基于以上考虑，我们选择在线程级别，使用TaskStatus描述任务的调度状态；在进程级别，则通过PCB中的停止标记和终止状态描述进程是否被信号停止、继续或退出。

状态	含义
Ready	任务已经准备好运行，可以进入ready queue，等待调度器选中
Running	任务正在某个CPU上运行
Blocked	任务正在等待某个事件，例如futex、pipe、socket、waitpid 等

Sleep	任务因定时器睡眠，例如nanosleep、clock_nanosleep
Zombie	任务已经退出，不会再被调度，等待后续清理

任务间的状态转换情况如下：

1. Ready ↔ Running: 当调度器从ready queue中选中一个任务时，该任务由Ready转换为Running，并开始在某CPU上运行。当任务主动让出CPU或被时钟中断抢占后，如果仍然可以继续运行，则会从Running重新变为Ready，并再次加入ready queue，等待后续调度。

2. Running ↔ Blocked: 当任务需要等待某个事件时，会从Running转换为Blocked，并切换回调度器。当等待事件发生后，内核将任务重新置为Ready，随后任务被调度器选中后再进入Running状态。

3. Running ↔ Sleep: 当任务执行定时睡眠系统调用时，会从Running转换为Sleep，并加入定时器队列。定时器到期后，内核将任务唤醒为Ready，随后调度器再次选中该任务，任务重新进入Running状态。

4. Running → Zombie: 当任务调用exit、线程函数返回，或者因致命信号退出时，任务会从Running转换为Zombie。Zombie表示任务已经结束，不会再被调度执行，只等待后续清理和资源回收。

5. Ready/Blocked/Sleep→Zombie: 在多线程进程中，如果进程被exit_group、SIGKILL等方式整体终止，其他线程即使当前不在Running状态，也可能被标记为Zombie。这样可以避免进程已经退出后，仍然存在可运行、阻塞或睡眠中的残留线程。

需要注意的是，我们并没有将可中断等待、不可中断等待、信号打断等语义直接设计为TaskStatus的枚举项。我们的想法是TaskStatus主要描述任务是否可被调度器选中，而信号投递、阻塞系统调用被打断、futex 唤醒等更细粒度的语义，由TCB中的线程级信号字段和阻塞唤醒字段配合实现。相关机制将在信号处理章节中进一步说明。

2.3 任务调度与任务执行

Kairix采用基于有栈线程的任务调度机制，每个用户线程都由一个TCB表示，并拥有独立的KernelStack、KContext和TrapFrame。其中，KernelStack保存线程在内核态执行时的调用栈，KContext保存内核上下文切换所需的寄存器状态，TrapFrame保存用户态陷入内核时的用户寄存器现场。

主要的调度循环则由run_tasks()控制，从ready queue选择TCB，若第一次执行，则通过task_entry()开始执行，紧接着激活用户空间，进入用户态。若该任务此前已经运行过，则会从上一次保存KContext的位置继续执行。

任务执行的核心流程包括：

1. **任务选择：**调度器从当前CPU的TaskManager中获取Ready状态的任务。
2. **执行前准备：**激活该任务所属进程的地址空间，
3. **内核上下文切换：**调度器切换KContext
4. **任务入口执行：**任务第一次运行时会进入task_entry()。
5. **用户态执行与陷入处理：**CPU陷入内核时用户态寄存器现场被保存到TrapFrame中。
6. **信号处理：**准备再次返回用户态前，内核会检查待处理信号。
7. **任务退出：**退出后，等待资源回收。

3 内存管理

3.1 物理内存管理

Kairix通过物理页分配器按页管理系统中的可用物理内存。可用物理内存是指物理内存中，去除内核的代码和数据、设备树等启动阶段保留区域之后，能够交给内核动态分配的内存区域。这些物理页主要用于存放页表、用户进程数据页、内核按页分配的数据结构、页缓存以及文件映射页面等。

Kairix当前使用的物理页分配器是StackFrameAllocator。该分配器以多个连续物理页区间作为基础，每个区间维护start、current和end三个页号。其中，current表示该区间中尚未分配部分的起始页号。分配新物理页时，分配器优先从回收栈recycled中取出已经释放的页；如果回收栈为空，则在各个可用区间中按页号递增分配。

FrameTracker是Kairix对物理页生命周期的RAII封装。创建FrameTracker时，内核会清零该物理页，避免旧数据泄露给新的使用者；当FrameTracker被释放时，其Drop实现会自动调用frame_dealloc将物理页归还给分配器。这种设计使得物理页的分配和释放既简单又安全，避免了手动管理内存的复杂性和潜在的内存泄漏风险。

```

1 pub struct FrameTracker {
2     ///
3     pub ppn: PhysPageNum,
4 }
5
6 impl FrameTracker {
7     ///Create an empty `FrameTracker`
8     pub fn new(ppn: PhysPageNum) -> Self {
9         // page cleaning
10         let bytes_array = ppn.get_bytes_array();
11         for i in bytes_array {
12             *i = 0;
13         }
14         Self { ppn }
15     }
16 }
17
18
19 impl Drop for FrameTracker {
20     fn drop(&mut self) {
21         frame_dealloc(self.ppn);
22     }
23 }

```


3.2 内核动态内存分配

Kairix通过伙伴内存分配算法为内核中的动态内存结构分配和释放内存，该算法的实现由Rust包buddy_system_allocator提供。Kairix使用的伙伴分配算法会将堆空间按照2的幂大小划分为不同层级的空闲块。当内核申请一块内存时，分配器会选择能够容纳该请求的最小空闲块；如果该块过大，则不断拆分为两个大小相同的伙伴块，直到得到合适大小的内存块。释放内存时，如果被释放块的伙伴块也处于空闲状态，分配器会将二者合并为更大的块，并继续尝试向上合并。该算法的优点是实现简单、分配速度快、碎片问题较小。

3.3 虚拟地址空间

操作系统通常有两种虚拟地址空间设计：一种是内核地址空间与用户地址空间共享同一套页表高地址映射，另一种是内核地址空间与用户地址空间隔离。地址空间隔离能够提高安全性，但会增加系统调用、异常处理和上下文切换时的页表切换成本。

Kairix为了降低上下文切换开销，采用类似早期Linux的共享内核映射设计。每个用户进程都有独立的用户地址空间，但其页表会从内核页表复制内核高地址部分映射。因此，当用户程序陷入内核后，内核可以继续在同一地址空间中访问内核代码、数据、物理内存直接映射区和MMIO区域，不需要额外切换到单独的内核页表。

Kairix将虚拟地址空间划分为用户空间和内核空间两部分。用户空间位于低地址区域，用于存放用户程序的ELF段、动态链接器、用户堆、用户栈、mmap区域、共享内存和trap context等。内核空间位于高地址区域，用于映射内核镜像、内核栈、内核堆所在静态区域、可用物理内存以及设备MMIO区域。

3.3.1 虚拟地址空间布局

Kairix当前支持RISC-V和LoongArch两种架构。两者的虚拟地址空间布局有所不同。

RISC-V架构中，虚拟地址空间布局如下：

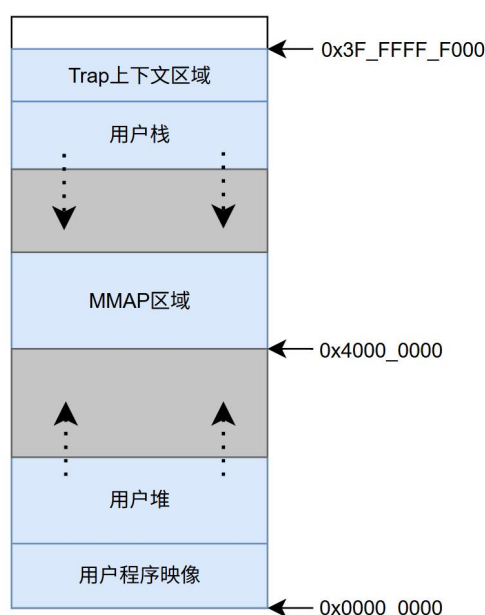
1. 用户地址空间：0x0000_0000_0000_0000 ~ 0x0000_003f_ffff_ffff
2. 内核地址空间：0xffff_ffc0_0000_0000 ~ 0xffff_ffff_ffff_ffff

LoongArch架构中，虚拟地址空间布局如下

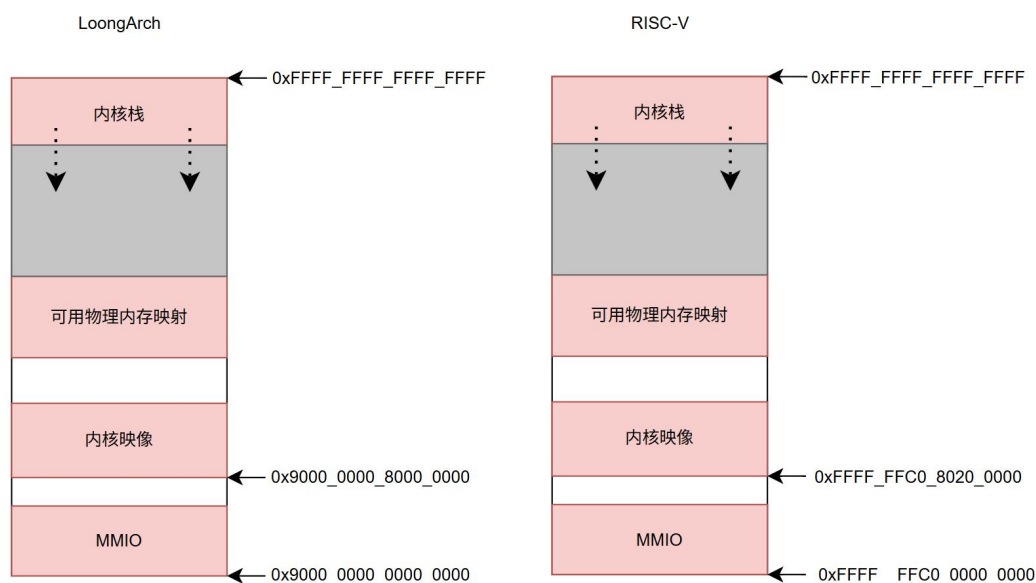
1. 用户地址空间：0x0000_0000_0000_0000 ~ 0x0000_003f_ffff_ffff
2. 内核地址空间：0x9000_0000_0000_0000 ~ 0xffff_ffff_ffff_ffff

其中，RISC-V 的内核虚拟地址偏移为0xffff_ffc0_0000_0000，LoongArch的内核虚拟地址偏移为0x9000_0000_0000_0000。内核访问物理内存时，通常通过“物理地址+内核虚拟地址偏移”得到对应的内核虚拟地址。该设计使内核可以用统一方式访问物理页内容、设备寄存器以及页表页。

需要注意的是，内核地址空间虽然覆盖很大的高地址范围，但并不表示所有地址都合法。实际可访问范围取决于内核页表中建立的映射、物理内存和MMIO区域。访问未映射区域仍会触发异常。



用户地址空间



内核地址空间

3.3.2 共享内核地址空间

在Kairix中，每个进程拥有独立的用户地址空间，同时共享同一套内核地址空间映射。进程切换时，调度器会切换当前任务所属进程的用户页表，使不同进程看到各自独立的用户虚拟地址空间；但页表中的内核高地址部分保持一致，因此所有进程陷入内核后都能够访问同一套内核代码、数据、物理内存映射和MMIO映射。

Kairix通过`UserVMSet::from_kernel`创建用户地址空间。该函数会新建用户页表，并将`KERNEL_VMSET`中的内核页表项复制到用户页表根中。这样，每个用户进程虽然拥有独立的低地址用户映射，但高地址内核映射与全局内核页表保持一致。

```

1  ///继承内核页表映射
2  pub fn from_kernel(kernel_vm_set: &KernelVMSet) -> Self {
3      trace!("from_kernel");
4      let page_table = PageTable::new();
5      page_table
6          .root()
7          .get_pte_array()
8          .copy_from_slice(&kernel_vm_set.page_table.root().get_pte_array()[..]);
9      Self {
10         page_table: page_table,
11         areas: Vec::new(),
12     }
13 }
```

该设计的优势有：

1. **异常处理路径简单：**用户程序发生系统调用、中断或异常后，CPU进入内核态时仍处于当前进程页表下。内核可以直接执行trap处理代码而无需先切换到单独的内核页表。
2. **内核访问用户空间更高效：**Kairix可以基于当前进程页表直接翻译用户虚拟地址，访问用户数据，避免为每次用户缓冲区访问额外建立临时内核映射。
3. **多架构统一：**LoongArch对内核部分区域使用直接映射窗口，RISC-V对这部分地址空间页表采用相同映射方式，地址空间布局统一。上层采用相同的接口实现页表的切换。

3.4 用户地址空间管理

Kairix通过`UserVMSet`、`UserMapArea`和`PageTable`三类核心数据结构管理用户地址空间。`UserVMSet`是对一个用户进程虚拟地址空间的整体抽象，内部包含一个页表实例和若干个用户虚拟内存区域；`UserMapArea`描述用户地址空间中一段连续虚拟地址范围；`PageTable`则负责维护虚拟页到物理页之间的实际映射关系。`UserVMSet`的基本结构如下：

```

1 pub struct UserVMSet {
2     pub page_table: PageTable,
3     pub areas: Vec<UserMapArea>,
4 }

```

其中，page_table表示该进程当前使用的页表，areas保存所有用户态VMA。

UserMapArea是用户地址空间中一段连续虚拟地址区域的描述。它不仅记录起止地址和权限，还记录该区域的类型、物理页、懒分配状态、写时复制状态、文件映射信息以及共享内存标识等。

```

1 pub struct UserMapArea {
2     pub va_range: VRange,
3     pub data_frames: BTreeMap<VirtPageNum, Arc<FrameTracker>>,
4     pub map_type: MapType,
5     pub map_perm: MapPermission,
6     pub area_type: UserMapAreaType,
7     pub cow_flag: bool,
8     pub lazy_flag: bool,
9     pub growdown_flag: bool,
10    pub map_file: Option<Arc<dyn File>>,
11    pub file_offset: usize,
12    pub flags: MmapType,
13    pub shmid: Option<usize>,
14 }

```

3.4.1 懒分配

Kairix在用户地址空间管理中大量使用懒分配策略。对于部分用户虚拟地址区域，内核在创建映射时并不立即分配物理页，也不一定立即建立页表项，而是先在UserVMSet中创建对应的UserMapArea，记录该区域的虚拟地址范围、权限、类型以及相关属性。这样，在逻辑上该地址范围已经属于用户进程，但只有当用户程序真正访问该地址时，内核才会通过缺页异常完成物理页分配和页表更新。

当用户程序首次访问该区域时，CPU 触发缺页异常。内核根据异常地址查找所属UserMapArea，如果该访问符合VMA权限要求，就分配物理页并更新页表。

Kairix当前主要在以下场景使用懒分配：

1. 用户堆：用户程序通过brk扩展堆时，内核主要扩展Heap类型UserMapArea的虚拟地址范围。实际物理页会在用户程序首次访问对应地址时分配。
2. 用户栈和growdown区域：用户栈区域支持按需分配物理页；当访问接近栈 边界时，内核还可以尝试扩展栈范围。
3. 匿名mmap：用户通过mmap申请匿名内存时，内核创建Mmap类型区域，但不会立即为整个映射区分配物理页。

4. 文件mmap：对文件映射区域，Kairix会记录map_file、file_offset和映射权限。首次访问时再按页从文件页缓存取得内容，或为MAP_PRIVATE映射复制私有页。

5. 共享内存：Shm类型区域也可以通过VMA记录共享内存ID和物理页关系，在需要时建立实际映射。

这种策略避免了对大范围物理内存的提前分配。例如，用户程序申请较大的mmap区域但只访问其中少数页面时，Kairix只会为实际访问过的页面分配物理内存，从而降低加载时间和内存占用。

3.4.2 写时复制

Kairix在fork场景中使用写时复制机制减少进程创建开销。传统fork如果直接复制父进程所有用户页，会带来大量内存分配和数据复制，而多数子进程会很快执行execve或只读取原有数据。写时复制允许父子进程暂时共享同一批物理页，只有当某一方尝试写入共享页时，内核才真正复制该页。

对于写时复制的区域，Kairix会在首次写入时触发缺页，再根据物理页的引用计数判断是否需要分配新物理页或者恢复物理页的写权限。

写时复制机制显著降低了fork的时间和空间成本。父子进程只为实际发生写入的页面付出复制代价。这既提升了进程创建性能，也减少了物理内存占用。

4 文件系统设计

4.1 虚拟文件系统

虚拟文件系统（Virtual File System, VFS）是内核对具体文件系统的抽象。它将下层具体文件系统统一为一个抽象文件系统，向用户程序提供一致的文件操作接口。

Kairix的VFS总体结构图如下：

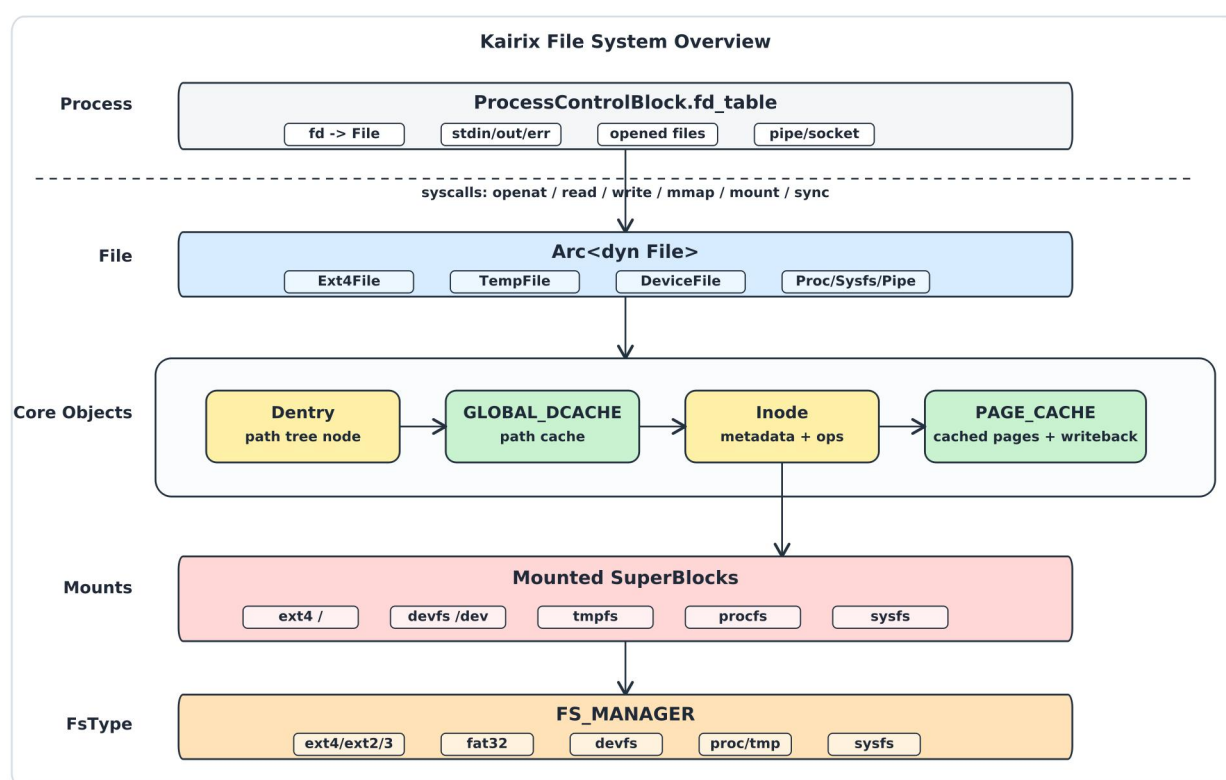


图5-1 VFS总览

这意味着，用户程序可以使用一组标准的系统调用来操作文件，无需关心文件实际存储于何种文件系统和存储介质，同时内核在处理文件时也只需与VFS层交互，无需关心具体的文件系统实现细节。

Kairix的文件系统以Linux文件系统为参考，充分结合了Rust的特性进行多态抽象，对VFS进行了设计和优化。目前Kairix支持多文件系统，主要支持ext4，并且基本支持Fat32。此外，Kairix还设计了很多非磁盘文件系统，包括但不限于procfs、tmpfs、devfs，设计理念符合“万物皆文件”的思想。

4.2 核心数据结构设计

Kairix的核心数据结构主要参考Linux，并且借鉴了Chronix的一部分设计，在此基础上进行了接口的优化，最终形成了Kairix的文件系统。

Kairix的核心数据结构包括：

1. FsType: 文件系统类型的抽象，一个FsType代表一个文件系统类型
2. SuperBlock: 具体的文件系统，因为一个文件系统可能会有多个文件系统实例，所以使用SuperBlock来代表每一个具体的文件系统实例。
3. Inode: 每一个Inode代表着一个独一无二的资源。
4. Dentry: 目录项，用Dentry管理路径到Inode的映射
5. File: 文件在进程中的映射，一个Inode在不同进程中会有多个File。

4.2.1 FsType

在Linux内核的虚拟文件系统（VFS）架构中，file_system_type是每种文件系统实现的注册入口，是连接内核VFS与具体文件系统实现的桥梁。在这个层级，Kairix参考了Chronix的设计，简化了Linux的设计，并实现一些特化。

```

1 pub struct FsTypeInner {
2     /// name of the file system type
3     name: String,
4     /// the super blocks
5     pub supers: Mutex<BTreeMap<String, Arc<dyn SuperBlock>>>,
6 }
7 pub trait FsType: Send + Sync {
8     /// get the base fs type
9     fn inner(&self) -> &FsTypeInner;
10    /// mount a new instance of this file system
11    fn mount(
12        &self,
13        name: &str,
14        parent: Option<Arc<dyn Dentry>>,
15        flags: MountFlags,
16        dev: Option<Arc<dyn BlockDevice>>,
17    ) -> SysResult<Arc<dyn Dentry>>;
18    /// shutdown a instance of this file system
19    fn kill_sb(&self) -> isize;
20    /// get the file system name
21    fn name(&self) -> &str {
22        &self.inner().name
23    }
24    /// use the mount path to get the super block
25    fn get_sb(&self, abs_mount_path: &str) -> Option<Arc<dyn SuperBlock>> {

```

```

26         self.inner().supers.lock().get(abs_mount_path).cloned()
27     }
28     /// get the static superblock
29     /// add a new super block
30     fn add_sb(&self, abs_mount_path: &str, super_block: Arc<dyn SuperBlock>)
31     {
32         self.inner()
33             .supers
34             .lock()
35             .insert(abs_mount_path.to_string(), super_block);
36     }

```

通过FsType的抽象，Kairix可以很快的将不同类型的文件系统进行挂载到现有的框架之下。

4.2.2 SuperBlock

超级块（SuperBlock）用于存储特定文件系统的信息，通常对应于存放在磁盘特定扇区中的文件系统超级块。超级块是对文件系统的抽象，即一个超级块对应于一个文件系统的实例。

```

1  /// the base of super block of all file system
2  pub struct SuperBlockInner {
3      /// the block device fs using
4      pub device: Option<Arc<dyn BlockDevice>>,
5      /// the root dentry
6      pub root: SpinNoIrqLock<Option<Arc<dyn Dentry>>>,
7      /// mount flags
8      pub flags: AtomicU32,
9  }

```

其中，

device：表示该文件系统所依赖的底层块设备。

root：代表该文件系统的根目录项。

Flags：代表挂载的当前的挂载属性

可以发现，Kairix的超级块做了简化设计，不像是其他内核的包含具体的文件类型的字段，而是通过FsType进行动态分发创建超级块，因此这里没有保存这个具体的文件类型的字段。

4.2.3 Dentry

目录项(Dentry)是文件系统中文件树结构的结点。它包含文件名和指向索引节点的指针。目录项用于将文件名映射到索引节点，从而实现文件系统的层次结构。每个目录项都包含一个文件名和一个指向对应索引节点的指针，目录项可以是文件或目录。

当用户程序访问某个路径时，内核并不会直接操作底层文件系统的inode，而是先通过路径解析得到Dentry，再找到其关联的Inode。这种设计使路径结构和文件元数据解耦，也使ext4、tmpfs、procfs、devfs等不同文件系统可以统一接入VFS路径树。同时，我们会使用Dcache来加快缓存路径的查询速度。

```

1 #[allow(unused)]
2 ///the detail of data in dentry
3 pub struct DentryInner {
4     /// Name.
5     pub name: String,
6     /// Parent dentry. This field is `None` if this dentry is the root of the
    filesystem.
7     pub parent: Option<Weak<dyn Dentry>>,
8     /// Children dentries.
9     pub children: Mutex<BTreeMap<String, Arc<dyn Dentry>>>>,
10    /// Inode that this dentry points to.
11    pub inode: Mutex<Option<Arc<dyn Inode>>>>,
12    /// Dentry before mount. `None` if this dentry has not been mounted.
13    pub mdentry: Mutex<Option<Arc<dyn Dentry>>>>,
14    /// Dentry bind mount. `None` if this dentry has not been bind-mounted.
15    pub bdentry: Mutex<Option<Arc<dyn Dentry>>>>,
16 }

```

其中：

1. name: 当前Dentry节点的名字。
2. parent: 当前节点的父节点。通过递归组合父节点路径和当前节点名字，可以得到完整路径。
3. children: 当前节点的子目录项集合，用于维护内存中的路径树。
4. inode: 当前Dentry映射到的Inode。
5. mdentry: 当某个目录被新的文件系统挂载覆盖时，用于保存原来的目录项，便于umount后恢复。
6. bdentry: 用于支持bind mount场景下的目录项转发和回退。

我们这样设计Dentry的字段目的是为了更方便挂载，在调研了其他组的内核的挂载实现之后，我们发现大家的挂载手段千奇百怪，部分内核采取的是传统设计，也就是维护一个外部

挂载表。我们认为这样方案缺少了一点美观，于是Kairix选择的是利用Dentry树的覆盖机制，利用mdentry和bdentry就能够达到简易的挂载设计，且高效。

4.2.4 Dcache

为了加快路径解析速度，Kairix在VFS层实现了全局目录项缓存GLOBAL_DCACHE。它维护从绝对路径到Dentry的映射，通过Dcache，内核可以直接通过路径命中对应Dentry，这个策略可以加速一些经常性事件：

```

1 struct LruMeta {
2     /// generation -> path, 按时间顺序找到最久未访问
3     order: BTreeMap<usize, String>,
4     /// path -> generation
5     path_to_gen: BTreeMap<String, usize>,
6     /// 单调递增访问计数器
7     next_gen: usize,
8 }
9
10 ///
11 struct DentryCacheInner {
12     dcache: BTreeMap<String, Arc<dyn Dentry>>,
13     lru: LruMeta,
14     pinned: BTreeSet<String>,
15 }
16
17 ///
18 pub struct DentryCache {
19     inner: SpinLock<DentryCacheInner>,
20     max_size: usize,
21 }
22 pub static ref GLOBAL_DCACHE: DentryCache = DentryCache::new(DCACHE_MAX_SIZE);

```

Kairix的DCache加入了LRU淘汰机制。当缓存超过容量上限时，会优先淘汰最久未访问且未被pin的路径。Pin住的一般都是重要挂载点，因此pin可以防止重要根目录被淘汰出缓存。

4.2.5 Inode

Inode是文件系统对象的元数据抽象。Kairix中的每个普通文件、目录、符号链接、设备文件、FIFO、socket等对象，都可以通过Inode描述其类型、大小、权限、时间戳和底层操作。这符合着我们团队的开发思想——“万物皆文件”。

```

1 #[allow(unused)]
2 /// Inode:i_ino
3 pub struct InodeInner {
4     pub ino: usize,

```

```

5  pub size: AtomicUsize,
6  pub nlink: AtomicUsize,
7  pub mode: InodeMode,
8  pub uid: AtomicUsize,
9  pub gid: AtomicUsize,
10 pub rdev: AtomicUsize,
11 pub atime_sec: AtomicI64,
12 pub atime_nsec: AtomicI64,
13 pub mtime_sec: AtomicI64,
14 pub mtime_nsec: AtomicI64,
15 pub ctime_sec: AtomicI64,
16 pub ctime_nsec: AtomicI64,
17 pub fs_flags: AtomicUsize,
18 pub punched_hole_pages: BTreeSet<usize>,
19 }

```

其中:

1. ino: inode编号
2. size记录文件大小
3. mode记录文件类型和权限
4. uid/gid用于记录所有者信息
5. rdev用于设备文件
6. 时间戳字段用于支持stat、utimensat等系统调用。
7. fs_flags支持类似immutable、append-only等文件系统标志,
8. punched_hole_pages用于记录稀疏文件中被打洞的页。

Inode中定义了读写、截断、同步、查找、删除、扩展属性等操作。不同文件系统会实现不同的inode类型。这样上层 只需要面向Arc<dyn Inode>操作, 而不需要关心对象来自ext4还是tmpfs。

4.2.6 File

文件是对可读写的数据流的抽象。它是文件系统中最常用的对象, 代表具体的文件或设备。每个文件对象与一个目录项关联, 进而与一个索引节点关联。

```

1  #[allow(unused)]
2  pub struct FileInner {
3      pub offset: usize,
4      pub dentry: Arc<dyn Dentry>,
5      pub flags: OpenFlags,
6  }

```

其中:

1. offset: 代表文件读写偏移量
2. Dentry: 保存dentry引用
3. Flags: 记录文件打开权限

可以发现，Kairix的File与其他内核的File有点区别，没有保存inode的引用。因为Kairix的文件系统设计是，File持有dentry引用，dentry再持有inode的结构，所以这里FileInner只保存dentry引用，不保存inode的引用

4.2.7 Fd Table

用户程序通过文件描述符来操纵文件。文件描述符是非负整数，内核使用它标识进程打开的文件，并允许用户程序通过它访问文件。每个进程都有一个文件描述符表，记录该进程的打开文件的文件描述符到文件对象的映射。

Kairix在每个PCB中单独维护文件描述符表的各个字段：

```
1 pub fd_table: Vec<Option<Arc<dyn File + Send + Sync>>>,&br/>2 pub fd_flags: Vec<u32>,&br/>3 /// 资源限制：文件大小上限  
4 pub rlimit_fsize: Rlimit64,&br/>5 /// 资源限制：单文件描述符最大数量  
6 pub rlimit_nofile: Rlimit64,
```

4.3 磁盘文件系统实现

4.3.1 FAT32 文件系统

FAT32，全称为File Allocation Table 32，是一种文件系统格式，用于在各种存储设备上存储和管理文件和目录。它是FAT文件系统的一个版本，最初由微软在1996年引入，主要是为了解决FAT16在处理大容量存储设备时的限制问题。FAT32文件系统在Windows操作系统以及许多其他设备和媒体中得到了广泛应用。

作为Windows设计的文件系统FAT32与UNIX风格文件系统差异较大，它本身没有完整的权限、硬链接、设备文件等语义。因此Kairix使用开源的rust- fatfs Rust库，在VFS层为FAT32补充了统一的inode、dentry和file抽象，使其能够被挂载到统一路径树中。不过由于竞赛要求的文件系统是ext4，因此，在当前Kairix中，FAT32更多用于兼容和扩展场景。正式根文件系统和主要测试环境仍以ext4为主。

4.3.2 ext4 文件系统

ext4是ext3的下一代标准，主要用于Linux操作系统。与前代文件系统相比，ext4在性能、可靠性、容量等方面有显著改进。

ext4是Kairix当前最主要的磁盘文件系统，也是默认根文件系统。Kairix使用开源库`lwext4_rust`作为底层ext4支撑，并在其上实现`Ext4FsType`、`Ext4SuperBlock`、`Ext4Dentry`、`Ext4Inode`和`Ext4File`。

4.4 非磁盘文件系统

在传统计算机系统中，文件系统通常用于管理磁盘等持久化存储设备上的数据，如Ext4、FAT32、NTFS等。但在现代操作系统中，文件系统的作用已经不再局限于磁盘数据管理。内核还会通过非磁盘文件系统（Non-Disk File Systems）向用户态暴露设备、进程信息、系统状态和临时内存对象等资源。

这类文件系统通常不依赖真实的物理存储介质，而是由内核在运行时动态维护或生成。它们在形式上仍然表现为目录和文件，与传统磁盘文件系统相比，非磁盘文件系统具有以下特点：

1. 不占用物理存储：数据通常存储在内存中，或由内核动态生成。
2. 动态内容：文件内容可能随系统状态实时变化。
3. 特殊用途：主要用于系统管理、调试、设备控制和进程间通信，而非持久化存储。
4. 高性能：由于不涉及磁盘I/O，访问速度极快（如`tmpfs`）。

4.4.1 procfs

procfs是进程文件系统，挂载在`/proc`目录下。它不占用磁盘空间，而是由内核动态生成内容，用于向用户态提供内核状态、进程状态和系统参数等信息。

Kairix中的procfs通过`ProcFsType`注册到VFS，并在启动时挂载到`/proc`。挂载完成后，内核会初始化一系列目录项和文件节点。这些文件大多没有真实的磁盘数据，而是在用户读取时由对应的File实现动态生成内容。

Kairix当前支持的procfs节点包括：

1. `/proc/meminfo`：系统内存使用情况。
2. `/proc/mounts`：当前挂载点信息。
3. `/proc/version`：内核版本信息。
4. `/proc/self`：指向当前进程相关信息。
5. `/proc/config.gz`：用于满足部分测试框架对内核配置文件的访问需求。

6. `/proc/cgroups`: 提供cgroup相关探测信息。
7. `/proc/sys/*`: 提供内核运行一些相关系统参数。

procfs的设计使得用户态程序可以像读取普通文件一样获取内核信息，这对于LTP等常见Linux工具的运行非常重要。

Kairix 特殊文件系统挂载结构

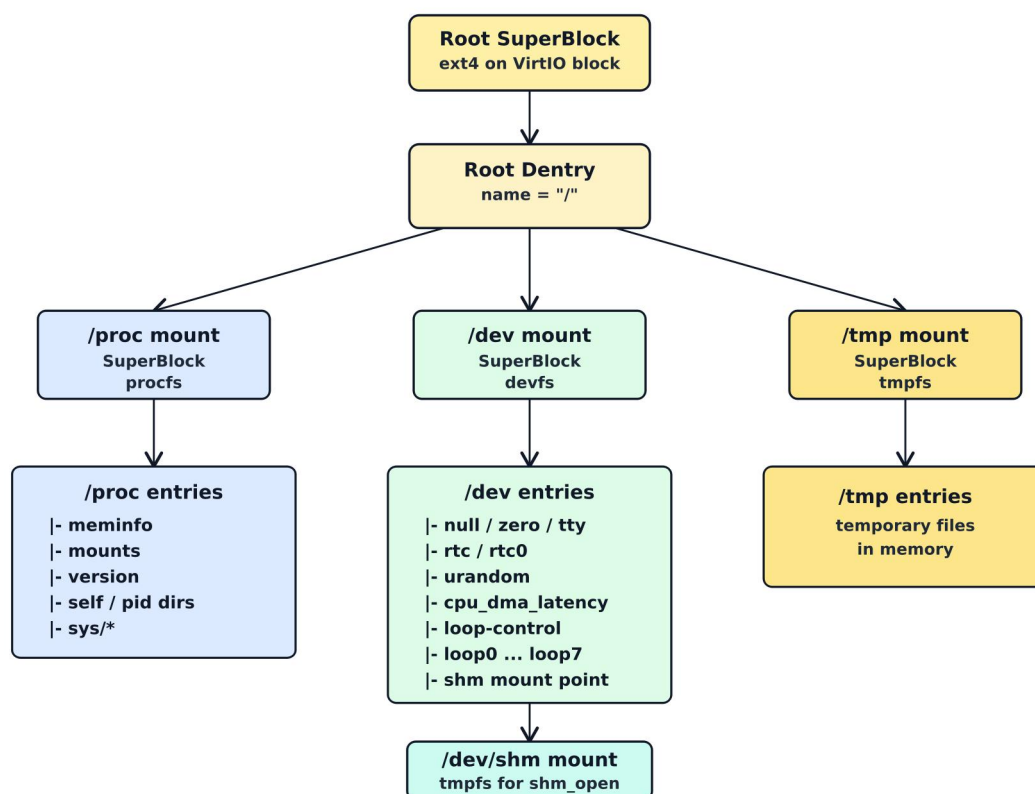


图5-2: 特殊挂载文件系统结构

4.4.2 devfs

devfs是设备文件系统，挂载在`/dev`目录下。它用于向用户态暴露设备节点，使用户程序可以通过标准文件接口与内核设备交互。

Kairix在启动时挂载devfs，并创建常用设备节点。当前主要支持：

1. `/dev/null`: 空设备，丢弃所有写入数据。
2. `/dev/zero`: 读取时返回全 0 数据。
3. `/dev/tty`: 终端设备，用作标准输入、标准输出和标准错误。
4. `/dev/rtc`、`/dev/rtc0`: 实时时钟设备。
5. `/dev/urandom`: 伪随机数设备。
6. `/dev/cpu_dma_latency`: 用于兼容用户态对CPU延迟控制接口的访问。

- 7. /dev/loop-control: loop设备控制节点。
- 8. /dev/loop0到/dev/loop7: loop块设备节点。
- 9. /dev/shm: 共享内存挂载点。

4.4.3 tmpfs

tmpfs是基于内存的临时文件系统。它不依赖底层磁盘设备，文件数据主要保存在内存和页缓存中。Kairix将tmpfs挂载在/tmp

目前/dev/shm的实现本质上是通过tmpfs进行实现的。

4.5 页缓存

在没有页缓存机制时，用户读写磁盘文件通常需要直接触发底层文件系统和块设备I/O。若一个文件被频繁访问，内核会反复从磁盘读取相同数据，造成大量重复I/O，性能开销较大。

为了解决这个问题，我们便引入了页缓存机制。Kairix在VFS层实现了统一的页缓存机制。页缓存以页为单位缓存文件内容，使文件读写尽可能在内存中完成。第一次读取文件时，内核从磁盘加载对应页并放入页缓存；之后再次访问相同位置时，可以直接命中缓存页，避免重复访问块设备。同时，Kairix的mmap文件映射也可以通过页缓存获得文件页对应的物理帧，从而将文件内容映射到用户地址空间中。

与部分内核中“每个inode持有一个PageCache”的设计不同，我们采用的全局页缓存，这种实现牺牲了一部分的生命周期管理的直观性，换来了统一的管理和回收。回收策略采用LRU和延迟写回。

4.5.1 Page

Kairix中的缓存页由Page描述：

```
1  ///
2  pub struct Page {
3      ///
4      pub frame: Arc<FrameTracker>,
5      ///
6      pub dirty: bool, //脏页标记!
7  }
```

Page包含两个核心字段：

1. frame：指向实际物理页帧的引用。
2. dirty：脏页标志。

Kairix的Page本身不保存文件偏移。文件偏移信息由PageCache的key维护，也就是：
(inode_id, page_id) -> Page，其中page_id表示该页在文件中的页号。这样可以让Page更简单，只负责描述缓存页本身

4.5.2 PageCache

```
1  ///
2  pub struct PageCache {
3      // Key: (inode_id, page_id)
4      cache: BTreeMap<usize, usize>, Arc<RwLock<Page>>>,
5      /// generation -> key, 用于按时间顺序找到最久未访问的页
6      lru_order: BTreeMap<usize, (usize, usize)>,
7      /// key -> generation, 用于 O(log n) 更新访问顺序
8      lru_gen: BTreeMap<usize, usize>,
9      /// 单调递增的访问计数器
10     next_gen: usize,
11     /// 最大页数
12     max_pages: usize,
13 }
```

其中：

1. cache：核心缓存表，维护(inode_id, page_id)到缓存页的映射。
2. lru_order：按照访问时间记录缓存页顺序，用于淘汰最久未使用的页。
3. lru_gen：记录每个缓存页当前的LRU generation。
4. next_gen：单调递增的访问计数器。
5. max_pages：页缓存最大容量，当前默认为4096页，约16MB。

4.6 其他数据结构

4.6.1 pipe

详细介绍见6.2进程间通信的管道机制。

5 进程间通信

进程间通信机制，包括信号、管道和共享内存。信号主要用于异步事件通知和进程控制；管道用于在进程之间传递字节流；共享内存则允许多个进程直接访问同一组物理页，从而实现高效的数据交换。这些机制分别覆盖了控制通知、流式通信和大块数据共享三类典型IPC场景。

5.1 信号机制

信号是一种轻量级的异步通知机制。Kairix中的信号既可以由一个进程发送给另一个进程，也可以由内核在异常、定时器、管道错误等场景下主动投递给进程。

我们的信号处理机制主要是参考的Linux的语义，支持1到64号信号，并为每个信号维护默认行为。信号的处理方式包括三种：默认处理、忽略处理和用户自定义处理函数。其中SIGKILL和SIGSTOP不能被捕获或忽略，用于保证内核始终可以强制终止或停止目标进程。

Kairix的信号处理整体流程如下图所示。

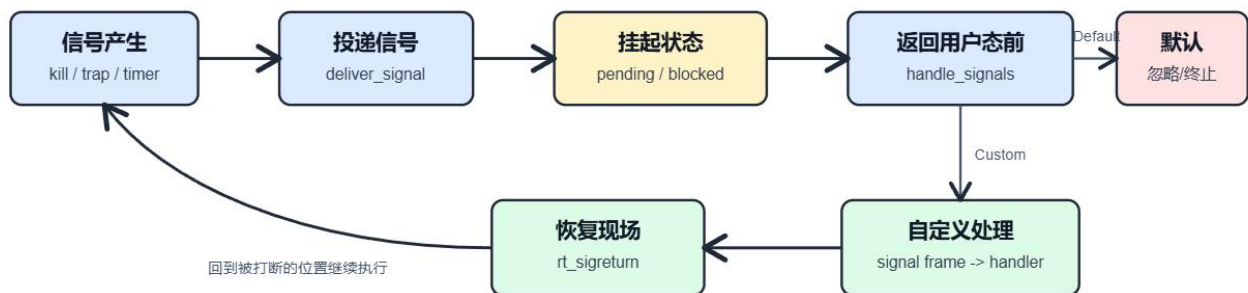


图6-1 信号处理流程

5.1.1 数据结构设计

```

1  /// 信号编号定义（使用 POSIX 标准全名）
2  /// 以结构体形式实现，支持 1..=64 的所有信号，包含实时信号。
3  #[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]
4  pub struct Signal(pub i32);
5
6  #[derive(Debug, Clone, Copy, PartialEq, Eq)]
7  pub struct SignalSet {
8      bits: u64,
9  }

```

Signal表示信号编号。

SignalSet表示信号集合，通过一个u64位图，可以高效的记录记录pending信号和blocked信号

其中，每个信号对应的处理方式我们都用SigAction描述。

```
1 /// 信号处理动作
2 #[repr(C)]
3 #[derive(Debug, Clone, Copy)]
4 pub struct SigAction {
5     /// 信号处理方式
6     pub sa_handler: SigHandler,
7     /// 在处理信号时要屏蔽的信号集
8     pub sa_mask: SignalSet,
9     /// 处理标志
10    pub sa_flags: u32,
11    /// 恢复函数地址 (musl 使用)
12    pub sa_restorer: usize,
13 }
```

其中：

1. sa_handler表示处理方式，可以是默认处理、忽略处理或用户自定义函数。
2. sa_mask表示执行该信号处理函数时需要额外屏蔽的信号集合。
3. sa_flags保存信号处理标志
4. sa_restorer用于指定用户态信号处理函数返回后的恢复入口。如果用户程序没有提供该地址，Kairix会使用内核提供的信号返回跳板页。

与上届的一等奖队伍Chronix不同，我们没有单独设计SigManager，而是将信号相关状态分别保存在进程控制块和线程控制块中。原因在于我们设计进程的时候就是PCB和TCB分离开的设计。并且这种设计更适合多线程语义：进程级信号可以被进程中的任意合适线程处理。

5.1.2 信号处理函数

在任务由内核态返回到用户态之前，往往需要执行信号处理函数，检查和处理挂起的信号，调用适当的信号处理程序或执行默认行为，确保进程能够正确响应和处理信号。这一机制是操作系统处理异步事件、进程控制和进程间通信的重要组成部分。

Kairix信号的处理方式本质上有三种：采用默认处理、选择忽略、采用用户自定义处理函数。与此同时，我们选择将信号处理方式抽象为SigHandler，这种处理方式可以简化设计，同时更加贴切Rust的语义，并在真正处理信号时根据SigHandler和SignalAction分别执行对应逻辑。

```
1 /// 信号处理方式枚举
2 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
3 pub enum SigHandler {
4     /// 默认处理 (SIG_DFL)
```

```

5     Default,
6     /// 忽略信号 (SIG_IGN)
7     Ignore,
8     /// 用户自定义函数
9     Custom(unsafe extern "C" fn(i32)),
10 }

```

其中：

1. Default表示采用该信号的默认行为。
2. Ignore表示忽略该信号。
3. Custom表示跳转到用户注册的信号处理函数。

对于Default类型的信号处理，Kairix进一步使用SignalAction描述不同信号的默认语义：

```

1 /// 信号默认处理动作
2 #[derive(Debug, Clone, Copy, PartialEq, Eq)]
3 pub enum SignalAction {
4     /// 终止进程
5     Terminate,
6     /// 忽略信号
7     Ignore,
8     /// 停止进程
9     Stop,
10    /// 继续进程
11    Continue,
12    /// 产生核心转储并终止
13    Core,
14 }

```

默认处理一共有五类：

1. Terminate：终止进程。例如大多数普通错误信号默认会导致进程退出。
2. Ignore：忽略信号。例如 SIGCHLD、SIGURG、SIGWINCH等信号默认可以被忽略。
3. Stop：停止进程。例如 SIGSTOP、SIGTSTP等信号会使进程进入停止状态。
4. Continue：继续运行被停止的进程，主要对应SIGCONT。
5. Core：终止进程并表示可产生core dump。例如SIGSEGV、SIGILL、SIGBUS、SIGFPE等异常类信号。

这些默认语义通过default_action()来进行指定。

```

1 /// 获取信号的默认处理动作
2 pub const fn default_action(self) -> SignalAction {
3     match self {
4         Self::SigChld | Self::SigCont | Self::SigUrg | Self::SigWinch
5     => SignalAction::Ignore,
6         Self::SigStop | Self::SigTstp | Self::SigTtin | Self::SigTtou
7     => SignalAction::Stop,
8         Self::SigQuit
9     | Self::SigIll
10    | Self::SigAbrt

```

```

9      | Self::SigTrap
10     | Self::SigFpe
11     | Self::SigSegv
12     | Self::SigSys
13     | Self::SigBus
14     | Self::SigXcpu
15     | Self::SigXfsz => SignalAction::Core,
16     _ => SignalAction::Terminate,
17   }
18 }

```

由此可以很清晰的了解Kairix的信号处理流程：当信号真正被处理时，Kairix会根据SigHandler进行分发：如果是Ignore，则清除pending状态；如果是Default，则根据default_action()执行对应默认动作；如果是Custom，则内核不会直接调用用户函数，而是在返回用户态前构造信号上下文并修改TrapFrame，使用户程序跳转到注册的handler中执行。

5.1.3 信号上下文管理

在上一小节，我们讲到如果是Custom，则内核不会直接调用用户函数，而是在返回用户态前构造信号上下文并修改TrapFrame，使用户程序跳转到注册的handler中执行。那么这里就涉及到一个设计问题。信号的上下文放在哪里？

部分作品如往届一等奖作品Titanix将上下文记录在内核态，待信号处理函数返回后再由内核恢复。这当然也可以，这种方式实现较简单，但在支持类POSIX信号语义时会遇到限制。例如，带有SA_SIGINFO语义的信号处理函数通常具有如下形式：

```
1 void handler(int sig, siginfo_t *info, void *ucontext);
```

其中ucontext指向用户可访问的上下文信息，用户态handler可以读取信号发生时的寄存器状态、信号掩码等内容。如果上下文只保存在内核对象中，内核就不能直接把该地址暴露给用户，否则会带来安全问题。因此，更合理的做法是：内核在用户栈上构造一份用户态可访问的上下文副本，让ucontext指向用户栈上的上下文区域。

因此，Kairix参考了Linux的设计，采用Linux风格的用户栈signal frame保存信号上下文。内核在处理自定义信号时，从当前TrapFrame中取出原始PC、通用寄存器和信号掩码，将其写入用户栈上的siginfo+ucontext布局中；当用户handler通过rt_sigreturn返回时，内核再从这段用户栈内存中恢复原来的TrapFrame，这样ucontext指向用户栈就非常安全。

```

1  // 统一在用户栈构建信号帧
2      const SIGINFO_SIZE: usize = 128;
3      const UCONTEXT_SIZE: usize = 960;
4      const SIGFRAME_SIZE: usize = SIGINFO_SIZE + UCONTEXT_SIZE + 8;
5
6      let sp = trap_cx[polyhal_trap::trapframe::TrapFrameArgs::SP];
7      let new_sp = sp.saturating_sub(SIGFRAME_SIZE);
8      let token = inner.vm_set.page_table.token();
9

```

```
10 let mut frame = [0u8; SIGFRAME_SIZE];
```

```
1 // 统一在用户栈构建信号帧
2     const SIGINFO_SIZE: usize = 128;
3     const UCONTEXT_SIZE: usize = 960;
4     const SIGFRAME_SIZE: usize = SIGINFO_SIZE + UCONTEXT_SIZE + 8;
5     const RESTORER_CODE: [u8; 8] = [0x0b, 0x2c, 0x82, 0x03, 0x00, 0x00,
0x2b, 0x00];
6     // const RESTORER_CODE: [u8; 8] = [0xff, 0xff, 0xff, 0xff, 0xff, 0xff,
0xff, 0xff];
7
8     let sp = trap_cx[polyhal_trap::trapframe::TrapFrameArgs::SP];
9     let new_sp = sp.saturating_sub(SIGFRAME_SIZE);
10    let token = inner.vm_set.page_table.token();
11
12    let mut frame = [0u8; SIGFRAME_SIZE];
```

5.1.4 信号的处理整体流程

在任务每一次从内核态返回用户态之前，Kairix都会调用handle_signals(ctx)检查并处理挂起信号。

具体流程如下：

1. 检查当前线程的pending_signals，并结合blocked_signals找出未被屏蔽的信号。如果线程级没有可处理信号，则继续检查进程级pending_signals。
2. 若存在多个pending信号，Kairix按照信号编号从小到大选择第一个可处理信号。
3. 获取该信号对应的SigAction，根据sa_handler判断处理方式。如果Ignore，则清除该信号的pending状态，完成处理。如果Default，则调用default_action()获取默认行为。如果Custom，内核在用户栈上构造signal frame。
4. 修改当前TrapFrame：将PC设置为用户注册的handler地址，通过寄存器传入信号编号、siginfo地址和上下文区域地址，并将返回地址设置为sa_restorer或内核提供的信号返回跳板。
5. 内核返回用户态，用户程序开始执行信号处理函数。
6. 用户态handler执行完成后，通过sa_restorer或信号返回跳板发起rt_sigreturn 系统调用，重新进入内核。
7. rt_sigreturn从用户栈signal frame中读取保存的上下文，并恢复原来的TrapFrame和信号屏蔽字。
8. 信号处理完成，用户程序回到被信号打断的位置继续执行。

5.2 管道机制

管道（Pipe）是一种典型的进程间通信机制，用于在进程之间传递字节流数据。它可以看作一个内核维护的FIFO缓冲区：写端（writer）向队尾写入数据，读端（reader）从队头读取数据。用户程序通过pipe系统调用创建管道，内核会返回两个文件描述符：一个指向管道读端，一个指向管道写端。读端文件对象只支持读取，写端文件对象只支持写入。

在进程间通信场景中，父进程通常先创建管道，再通过fork创建子进程。由于子进程会继承父进程的文件描述符表，因此父子进程的fd表项可以指向同一组pipe文件对象。随后，一个进程关闭不需要的一端并向写端写入数据，另一个进程从读端读取数据，从而实现数据流在进程之间的传递。

Kairix中的pipe底层由共享的PipeRingBuffer实现。

```
1 #[derive(Copy, Clone, PartialEq)]
2 enum RingBufferStatus {
3     Full,
4     Empty,
5     Normal,
6 }
7 pub struct PipeRingBuffer {
8     pages: Vec<Option<PipePage>>,
9     capacity: usize,
10    head: usize,
11    tail: usize,
12    status: RingBufferStatus,
13    read_end: Option<Weak<Pipe>>,
14    write_end: Option<Weak<Pipe>>,
15    read_waiters: VecDeque<Arc<TaskControlBlock>>,
16    write_waiters: VecDeque<Arc<TaskControlBlock>>,
17    poll_waiters: VecDeque<Arc<TaskControlBlock>>,,
18 }
```

Kairix Pipe 通信核心结构

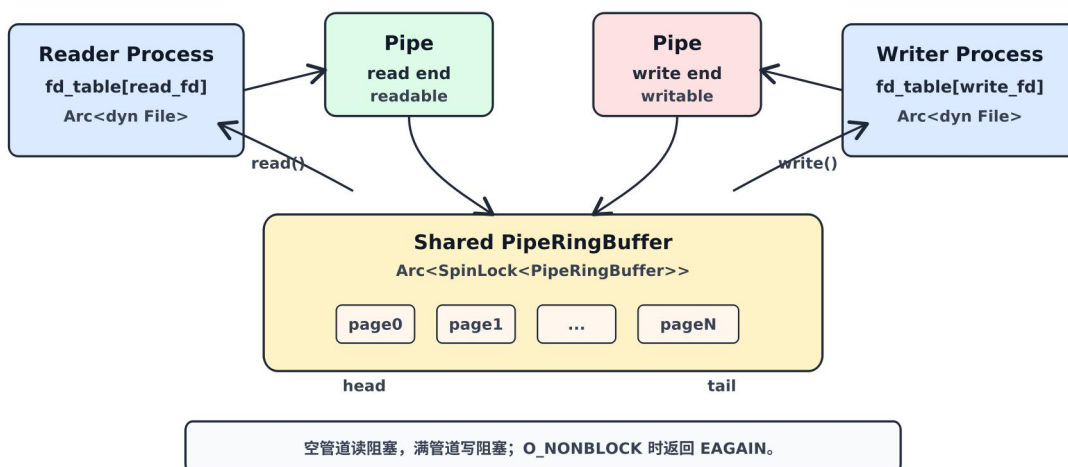


图6-2 Pipe通信结构图

5.3 共享内存机制

共享内存是一种高效的进程间通信机制。与管道这类字节流通信不同，共享内存允许多个进程将同一组物理页映射到各自的虚拟地址空间中，从而直接通过内存读写交换数据，避免了频繁的数据拷贝。

Kairix实现了完整的System V共享内存机制，允许多个进程共享同一块物理内存区域，实现高效的数据交换。

5.3.1 共享内存数据结构

Kairix使用全局的SHM_STATE管理所有共享内存段。每个共享内存段由ShmSegment描述：

```
1  /// A shared memory segment descriptor
2  struct ShmSegment {
3      /// Key (IPC_PRIVATE = 0 means private)
4      key: i32,
5      /// Size in bytes (page-aligned)
6      size: usize,
7      /// Number of pages
8      num_pages: usize,
9      /// Permission mode
10     mode: i32,
11     /// Physical pages backing this segment
12     pages: Vec<Arc<FrameTracker>>,
13     /// Number of current attaches
14     shm_nattch: usize,
15     /// Creator PID
16     shm_cpid: usize,
17     /// Last operation PID
18     shm_lpid: usize,
19     /// Marked for destruction (IPC_RMID)
20     destroyed: bool,}
```

其中：

1. key：共享内存的键值，用于支持具名共享内存；
2. size：共享内存段大小，Kairix会将其按页对齐。
3. num_pages：共享内存段占用的页数。
4. mode：权限信息。
5. pages：共享内存真正对应的物理页。
6. shm_nattch：当前挂接到该共享内存段的进程数量。
7. shm_cpid：创建该共享内存段的进程PID。
8. shm_lpid：最近一次操作该共享内存段的进程PID。

9. destroyed: 该共享内存段是否已被标记删除。

全局共享内存状态由ShmState描述:

```
1 /// Global shared memory state
2 struct ShmState {
3     /// Next shmid to assign
4     next_id: usize,
5     /// Map from shmid to segment
6     segments: BTreeMap<usize, ShmSegment>,
7     /// Map from key to shmid (for key-based lookup)
8     key_to_id: BTreeMap<i32, usize>,
9 }
```

其中:

1. Segments 用于通过shmid查找共享内存段
2. key_to_id 用于支持通过key查找已经存在的共享内存。

在调研其他队伍的实现时,我们发现部分内核采用懒分配的实现方式,这种实现方式节省内存。但是我们考虑到如果采用懒分配的话,并发的时候,如果多个进程同时访问同一个未分配页,那么共享页创建就可能产生竞态。而我们团队的目标是稳定,因此Kairix在shmget时就把所有物理页分配好,牺牲一点内存按需性,换来共享语义更直接、实现更稳。

5.3.2 共享内存系统调用

Kairix提供了四个关键的共享内存系统调用:

1. 创建/获取共享内存 (**sys_shmget**): 根据指定的key和大小创建新的共享内存段,或获取已经存在的共享内存段。Kairix支持IPC_PRIVATE和具名key两种模式,创建时会按页对齐大小,并为该段分配对应数量的物理页,最后返回共享内存标识符shmid。
2. 连接共享内存 (**sys_shmat**): 将指定共享内存段映射到当前进程的虚拟地址空间中。Kairix会创建Shm类型的VMA,并把共享内存段中的物理页映射到当前进程页表。成功后,进程可以直接通过返回的虚拟地址访问共享数据。
3. 分离共享内存 (**sys_shmdt**): 从当前进程地址空间中解除共享内存映射。Kairix会取消对应页表映射,移除Shm类型的VMA,并减少共享内存段的挂接计数。
4. 控制共享内存 (**sys_shmctl**): 用于查询、修改或删除共享内存段。对于IPC_RMID, Kairix会先将共享内存段标记为删除;当所有进程都分离该段后,系统才真正回收相关资源。

6 系统调用

6.1 系统调用接口

系统调用是操作系统提供给用户态程序访问内核服务的接口。Kairix提供Linux风格的系统调用接口，使用户程序可以通过统一方式请求文件系统、进程管理、内存管理、网络、信号等内核功能。

当应用程序执行系统调用时，会通过架构相关指令陷入内核。在RISC-V64上，用户程序使用`ecall`指令，系统调用号放在`a7`寄存器中，参数依次放在`a0`到`a5`中，返回值放在`a0`中。在LoongArch64上，用户程序使用`syscall 0`指令，系统调用号同样放在`$a7`，参数放在`$a0`到`$a5`，返回值放在`$a0`。

CPU接收到系统调用指令后，会从用户态切换到内核态，并进入Kairix的`trap`处理流程。内核根据保存下来的`TrapFrame`读取系统调用号和参数，调用统一的`syscall()`分发函数。处理完成后，内核将返回值写回返回寄存器，并恢复用户态上下文。

6.2 用户态与内核态切换

异常、中断和系统调用都会触发`trap`，产生从用户态到内核态的控制流切换。在`trap`这方面，我们团队复用了开源库，做出了优化，Kairix通过`polyhal`和`polyhal-trap`抽象不同架构的`trap`细节，使RISC-V64和LoongArch64能够复用大部分`trap`处理逻辑。

在RISC-V64上，用户态`trap`进入后，汇编入口会保存用户态通用寄存器、`sstatus`和`sepc`，并切换到当前任务的内核栈，然后调用Rust编写的`trap_handler`。处理完成后，内核通过恢复例程恢复用户寄存器、状态寄存器和PC，最后执行返回用户态的指令。

Kairix的用户上下文主要保存在`TrapFrame`中。以RISC-V64为例，其核心内容包括：

```
1 #[repr(C)]
2 #[derive(Clone)]
3 // 上下文
4 pub struct TrapFrame {
5     pub x: [usize; 32], // 32 个通用寄存器
6     pub sstatus: Sstatus,
7     pub sepc: usize,
8     pub fsx: [usize; 2],
9 }
```

其中，x保存通用寄存器，sstatus保存特权级和中断相关状态，sepc保存trap发生时的用户态PC。通过TrapFrameArgs抽象系统调用号、参数和返回值的位置，减少架构差异对syscall分发逻辑的影响。

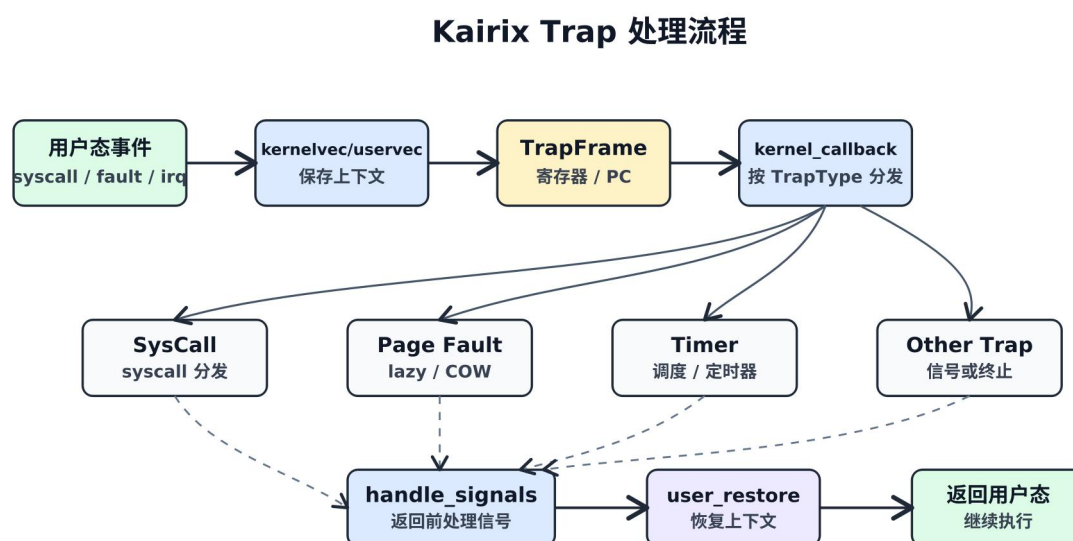


图7-1 Trap处理流程

6.2.1 陷阱处理机制

进入内核态后，Kairix通过trap_handler处理来自用户态的异常和中断。常见类型包括：

1. 系统调用：当trap类型为SysCall时，读取系统调用号和参数，进行系统调用处理。
2. 页错误：包括取指、读、写页错误。进入内存管理模块处理懒分配、COW、mmap、用户栈扩展等情况。若无法处理，则向进程投递SIGSEGV或SIGBUS。
3. 非法指令：当用户程序执行非法指令时，内核向当前进程投递SIGILL。

目前，Kairix支持的中断类型有：

1. 定时器中断：内核设置下一次定时器中断，检查alarm/itimer，必要时投递SIGALRM，并触发任务调度。
2. 其他中断或异常：根据具体trap类型进入对应处理逻辑，无法恢复时终止当前任务或进入内核错误处理。

6.2.2 内核态到用户态

trap处理完成后，Kairix会恢复用户态上下文并返回用户态。以RISC-V64为例，恢复过程会重新设置sscratch，恢复sstatus、sepc和通用寄存器，最后执行sret。

6.3 系统调用处理

我们选择同步系统调用处理模型，主要是面向实现稳定性和调试便利性。相比异步系统调用框架，同步模型的控制流更加直接，便于与Kairix现有调度器、等待队列和信号机制结合。

用户程序陷入内核后，trap_handler直接调用统一的syscall()函数进行分发：

```
1 TrapType::SysCall => {
2     // jump to next instruction anyway
3     ctx.syscall_ok();
4     _set_sum_bit();
5     let args = ctx.args();
6     // get system call return value
7     let syscall_id = ctx[TrapFrameArgs::SYSCALL];
8     let result = syscall(syscall_id, [
9         args[0], args[1], args[2], args[3], args[4], args[5],
10    ]);
11    ...
12 }
```

这种设计保持了系统调用实现的直接性，同时仍能通过调度器支持阻塞等待和并发运行。对于被信号打断的阻塞系统调用，Kairix会返回EINTR；对于带有SA_RESTART 标志的部分信号，则尽量避免打断阻塞系统调用，以提高Linux程序兼容性。

6.4 系统调用分发与错误处理

Kairix通过统一的syscall()函数分发系统调用。该函数根据系统调用号进行match，并调用对应模块中的实现函数。

系统调用统一返回SyscallResult，成功时返回Ok(value)，失败时返回Err(errno)。trap处理逻辑会将错误转换为Linux风格的负错误码，对于尚未支持的系统调用，Kairix返回ENOSYS：

```
1 pub type SyscallResult = Result<usize, SysError>;
2
3 match result {
4     Ok(val) => ctx[TrapFrameArgs::RET] = val,
5     Err(errno) => ctx[TrapFrameArgs::RET] = (-(errno.code() as
6         isize)) as usize,
7 }
```

6.5 系统调用列表

Kairix实现了大量Linux风格系统调用，主要包括以下类别：

类别	代表系统调用	功能描述
进程与线程管理	1 <code>clone, clone3, execve, exit, exit_group, wait4, waitid, getpid, gettid, getppid</code>	进程和线程创建、执行、退出、等待
内存管理	1 <code>brk, mmap, munmap, mprotect, msync, madvise, mlock, munlock</code>	虚拟内存映射、堆管理、权限和同步
文件系统	1 <code>openat, openat2, close, read, write, readv, writev, lseek, fstat, statx</code>	文件打开、读写、状态查询
目录与路径	1 <code>mkdirat, chdir, getcwd, unlinkat, getdents64, renameat2, linkat, symlinkat, readlinkat</code>	目录、路径和链接管理
文件系统挂载	1 <code>mount, umount2, sync, fsync, syncfs, open_tree, move_mount, fsopen, fsmount</code>	文件系统挂载、卸载和同步
网络通信	1 <code>socket, bind, listen, accept, connect, sendto, recvfrom, setsockopt, getsockopt, socketpair, shutdown</code>	socket 创建、连接和数据传输
信号处理	1 <code>rt_sigaction, rt_sigprocmask, rt_sigsuspend, rt_sigtimedwait, rt_sigreturn, kill, tkill, tgkill</code>	信号设置、屏蔽、等待、发送和返回
时间管理	1 <code>clock_gettime, clock_getres, clock_nanosleep, times, setitimer, getitimer, timerfd_create</code>	时间获取、休眠和定时器
同步与IPC	1 <code>futex, shmget, shmat, shmdt, shmctl, eventfd2</code>	futex、共享内存和事件通知
I/O控制	1 <code>ioctl, fcntl, ppoll, pselect6, epoll_create1, epoll_ctl, epoll_pwait</code>	设备控制和I/O多路复用
用户与权限	1 <code>getuid, geteuid, getgid, getegid, setuid, setgid, umask, capget, capset</code>	用户身份和权限管理
系统信息	1 <code>uname, sysinfo, syslog, getrusage, prlimit64</code>	系统信息和资源限制
兼容接口	1 <code>inotify, fanotify, landlock, bpf, userfaultfd, io_uring_setup</code>	面向Linux测试的兼容接口

7 网络模块

7.1 概述

Kairix的网络模块是一个完全自主实现的轻量级TCP/IP协议栈，支持IPv4、ARP、ICMP、UDP和TCP等核心协议，并提供了与POSIX兼容的套接字接口。网络模块采用清晰的分层架构，自底向上包括设备层、链路层、网络层、传输层和套接字层，各层之间通过统一的数据包缓冲区和trait接口进行交互，降低了模块间的耦合度，便于后续的扩展和维护。

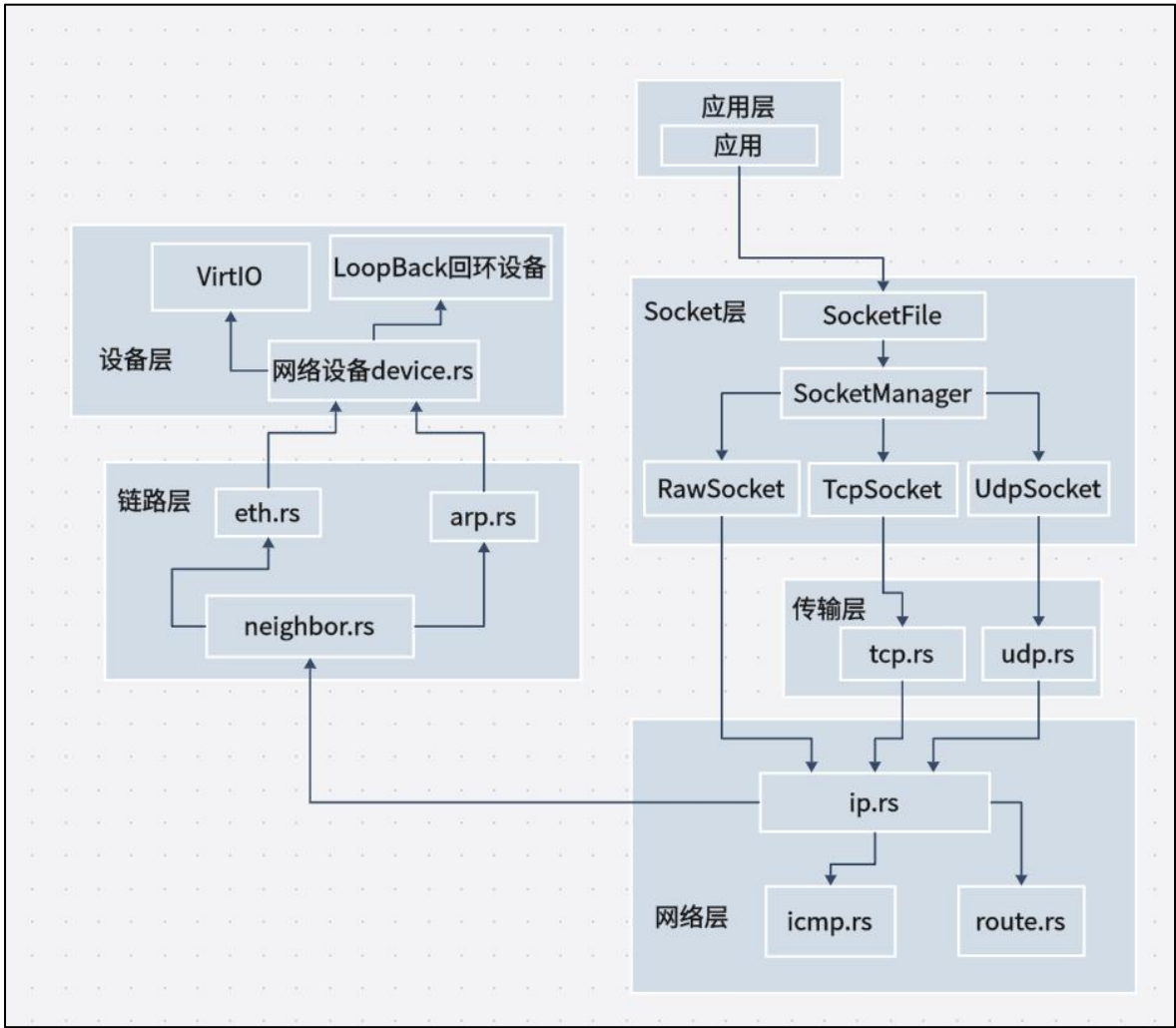


图8-1网络层总览图

7.2 数据包缓冲区

网络模块中所有数据包均以Skb结构为载体，采用连续内存缓冲区并借助起始偏移data_start和结束偏移data_end来标识有效数据区域。这种设计允许协议栈在逐层封装或剥离头部时，仅通过调整偏移量即可完成，避免了频繁的数据拷贝。同时，Skb关联了所属的网络设备，方便在接收和发送路径中追溯数据包的来源或去向：

```
1 #[allow(unused)]
2 /// 网络数据包缓冲区
3 pub struct Skb {
4     /// 数据缓冲区（连续内存）
5     pub data: Vec<u8>,
6     /// 数据区域的起始偏移（包含所有协议头）
7     pub data_start: usize,
8     /// 数据区域的结束偏移
9     pub data_end: usize,
10    /// 关联的网络设备
11    pub dev: Option<Arc<dyn NetDevice>>,
12 }
```

Skb提供了一些操作的方法，push用于在数据区前添加头部空间，put用于在尾部追加负载，pull用于移除已处理的头部，而reserve_head和reserve_tail则能够动态扩充缓冲区的容量以适应不同大小的协议头：

```
1 impl Skb {
2     pub fn push(&mut self, size: usize) -> Option<&mut [u8]>
3     pub fn put(&mut self, size: usize) -> Option<&mut [u8]>
4     pub fn pull(&mut self, size: usize) -> Option<&[u8]>
5 }
```

7.3 设备层

设备层是网络模块的最底层，负责抽象各类物理或虚拟网络设备，并向上层提供统一的发送和接收接口。该层通过NetDevice trait来规范所有设备的行为，其中包括设备名称、MTU、状态标志、MAC地址和IP地址的获取，以及数据包发送函数hard_start_xmit和接收回调注册函数set_rx_handler：

```
1 pub trait NetDevice: Send + Sync {
2     fn name(&self) -> &str;
3     fn mtu(&self) -> u16;
4     fn flags(&self) -> NetDeviceFlags;
5     fn hard_start_xmit(&self, skb: Skb) -> Result<(Skb, u32, u16), &'static str>;
6     fn set_rx_handler(&self, handler: Box<dyn Fn(Skb) + Send + Sync>);
7     fn poll_rx(&self) {}
8     fn mac_addr(&self) -> [u8; 6];
9     fn ip_addr(&self) -> u32;
10 }
```

7.4 链路层

链路层主要处理以太网帧的封装与解析，以及地址解析协议ARP。ethernet模块定义了标准的以太网帧头结构，当设备接收到数据包时，该模块会检查帧长度合法性，提取以太网类型，并根据类型值将数据包分发给相应的上层处理模块，IP 模块或ARP 模块。

ARP模块负责维护一个全局的IP到MAC地址映射缓存，关联对应的网络设备。当上层需要发送IP数据包但尚未知道目的MAC时，邻居层会调用ARP模块的查找接口：若缓存命中则直接返回MAC地址；若未命中，则构造ARP请求包，通过设备发送出去，并短暂轮询等待对应的ARP回复。

收到ARP数据包时，模块解析其中的操作码和地址字段，若为请求且目标IP为本机，则发送ARP回复；若为回复，则更新缓存。

```
1 static ARP_CACHE: Mutex<BTreeMap<u32, ArpEntry>> = Mutex::new(BTreeMap::new());
2 pub fn arp_lookup(ip: u32) -> Option<[u8; 6]>
3 pub fn arp_request(ip: u32, dev: Arc<dyn NetDevice>) -> Result<(), &'static
   str>
```

7.5 网络层

网络层的核心是IPv4协议，该模块负责IP数据包的接收、校验、路由和发送。

接收时，先检查IP头部长度和版本字段，确保为IPv4，接着计算头部校验和以验证完整性，然后提取源地址和目标地址。通过全局维护的本地IP地址列表LOCAL_IPS，模块判断目标地址是否属于本机，若属于，则根据协议字段（ICMP为1，TCP为6，UDP为17）将数据包分发给对应的传输层模块；若目标不是本机且未开启转发，则丢弃数据包。

```
1 pub fn ip_rcv(mut skb: Skb) -> Result<(Skb, u32, u16), &'static str> {
2     let ip_header = unsafe { &*(skb.data().as_ptr() as *const Ipv4Header) };
3     if ip_header.version() != 4 { return Err("Not IPv4"); }
4     let src_addr = ip_header.src_addr();
5     let dst_addr = ip_header.dst_addr();
6     if is_local_ip(dst_addr) {
7         skb.pull(ip_header.ihl() as usize);
8         match ip_header.protocol {
9             1 => icmp_rcv(skb, src_addr, dst_addr),
10            6 => tcp_rcv(skb, src_addr, dst_addr),
11            17 => udp_rcv(skb, src_addr, dst_addr),
12            _ => Err("Unsupported protocol"),
13        }
14     } else {
15         Err("Not for local")
16     }
17 }
```


发送时，`ip_queue_xmit`函数负责构造IP头部，设置版本、头部长度、服务类型、总长度、标识、标志片偏移、TTL（固定为64）和协议字段，然后计算校验和，随后调用路由查找函数确定输出设备和下一跳IP地址。路由表`route.rs`采用最长前缀匹配算法，维护了一系列路由条目，每个条目包含目标网络、掩码、网关和输出设备。查找返回的设备及下一跳信息被传递给邻居层`neighbor.rs`，该层先尝试通过 ARP 解析下一跳的MAC地址，若解析成功则在Skb头部封装以太网帧头，然后调用设备的发送函数完成最终的数据发送。若ARP解析失败，则返回错误并允许上层重试。

```
1 pub fn ip_queue_xmit(mut skb: Skb, src: u32, dst: u32, protocol: u8) ->
  Result<...> {
2     skb.reserve_head(core::mem::size_of::<Ipv4Header>());
3     let ip_header = unsafe { &mut *(skb.push(...).as_mut_ptr() as *mut
  Ipv4Header) };
4     ip_header.set_version_ihl();
5     ip_header.ttl = 64;
6     ip_header.protocol = protocol;
7     ip_header.src_addr = src.to_be();
8     ip_header.dst_addr = dst.to_be();
9     let (dev, nexthop) = route_lookup(dst)?;
10    neighbour_output(skb, nexthop, dev)
11 }
```

此外，ICMP协议`icmp.rs`作为IP的辅助协议，实现了Echo请求和回复功能，当收到Echo Request时，交换源和目的IP，将类型改为Echo Reply，重新计算校验和，然后通过IP层回送，从而支持Ping操作。

7.6 传输层

传输层提供了面向连接的TCP协议和无连接的UDP协议，分别实现了可靠的字节流传输和高效的数据报传输。

7.6.1 UDP

UDP套接字结构维护本地地址（IP和端口）以及可选的远程地址，接收队列以VecDeque存储收到的数据包及其来源信息，并设有接收缓冲区容量限制以防止内存溢出。绑定端口时若指定端口为0，则从全局临时端口范围中自动分配。

发送时构造UDP头部（源端口、目的端口、长度，校验和置零），通过IP层发送：

```
1 pub fn send_to(&self, data: &[u8], dst_addr: u32, dst_port: u16) ->
  SysResult<usize> {
2     let src = self.ensure_local()?;
3     send_udp_packet(src, data, dst_addr, dst_port)?;
4     Ok(data.len())
5 }
```



```

6 pub fn send_udp_packet(
7     src: (u32, u16),
8     data: &[u8],
9     dst_addr: u32,
10    dst_port: u16,
11 ) -> SysResult<(Skb, u32, u16)> {
12     if is_loopback_addr(dst_addr) {
13         return send_udp_loopback(src, data, dst_port);
14     }
15
16     let total_len = data.len() + UdpHeader::size();
17     let headroom = EthernetHeader::size() + core::mem::size_of::<Ipv4Header>();
18     let mut skb = Skb::with_headroom(headroom, total_len);
19     let udp_header = unsafe {
20         &mut *(skb.put(UdpHeader::size())
21             .ok_or(SysError::ENOMEM)?
22             .as_mut_ptr() as *mut UdpHeader)
23     };
24     udp_header.src_port = src.1.to_be();
25     udp_header.dst_port = dst_port.to_be();
26     udp_header.len = (total_len as u16).to_be();
27     udp_header.checksum = 0;
28     skb.put(data.len())
29         .ok_or(SysError::ENOMEM)?
30         .copy_from_slice(data);
31     ip_queue_xmit(skb, src.0, dst_addr, 17)
32         .map_err(|_| SysError::ENETUNREACH)?;
33     Ok((Skb::new(0), src.0, src.1))
34 }

```

接收时根据目的端口查找已注册的套接字，优先匹配完整的四元组（源IP、源端口、目的IP、目的端口），其次匹配仅绑定端口且未指定远程地址的套接字，成功则将数据包入队并唤醒等待的接收任务。

```

1 pub fn udp_rcv(mut skb: Skb, src_ip: u32, _dst_ip: u32) -> Result<(Skb, u32,
2     u16), &'static str> {
3     if skb.len() < UdpHeader::size() {
4         return Err("UDP packet too short");
5     }
6     let udp_header = unsafe { &*(skb.data().as_ptr() as *const UdpHeader) };
7     let dst_port = u16::from_be(udp_header.dst_port);
8     let src_port = u16::from_be(udp_header.src_port);
9
10    if let Some(socket) = lookup_udp_socket(dst_port, src_ip, src_port) {
11        skb.pull(UdpHeader::size());
12
13        let sock = socket.lock();
14        let payload_len = skb.len();
15        if sock.can_receive(payload_len) {
16            sock.enqueue(skb, src_ip, src_port);
17            sock.wake();
18            log::debug!("UDP: delivered packet to port {}", dst_port);
19        } else {
20            log::warn!("UDP: receive buffer full, dropping packet");
21        }
22        Ok((Skb::new(0), src_ip, src_port))
23    } else {
24        log::debug!("UDP: no socket for port {}", dst_port);
25        Err("No socket")
26    }
27 }

```

全局端口表UDP_SOCKETS维护了端口到套接字的映射，支持多路复用。

7.6.2 TCP

TCP 模块实现了完整的状态机，包括Closed、Listen、SynSent、SynReceived、Established、FinWait1、FinWait2、CloseWait、LastAck等状态，状态转换如下图所示，严格遵循RFC 793规范。每个TCP套接字拥有独立的发送序号和接收序号，初始序号由全局原子变量NEXT_ISS递增分配。监听套接字注册在全局LISTENERS表中，按（IP，端口）索引；已建立的连接则注册在CONNECTIONS表中，按（源IP，源端口，目的IP，目的端口）四元组索引，以便快速定位。

为提高并发性能，TCP模块在等待数据或连接时通过注册唤醒器Waker与异步调度器协作，使得任务可以挂起而不占用CPU，直至事件发生。

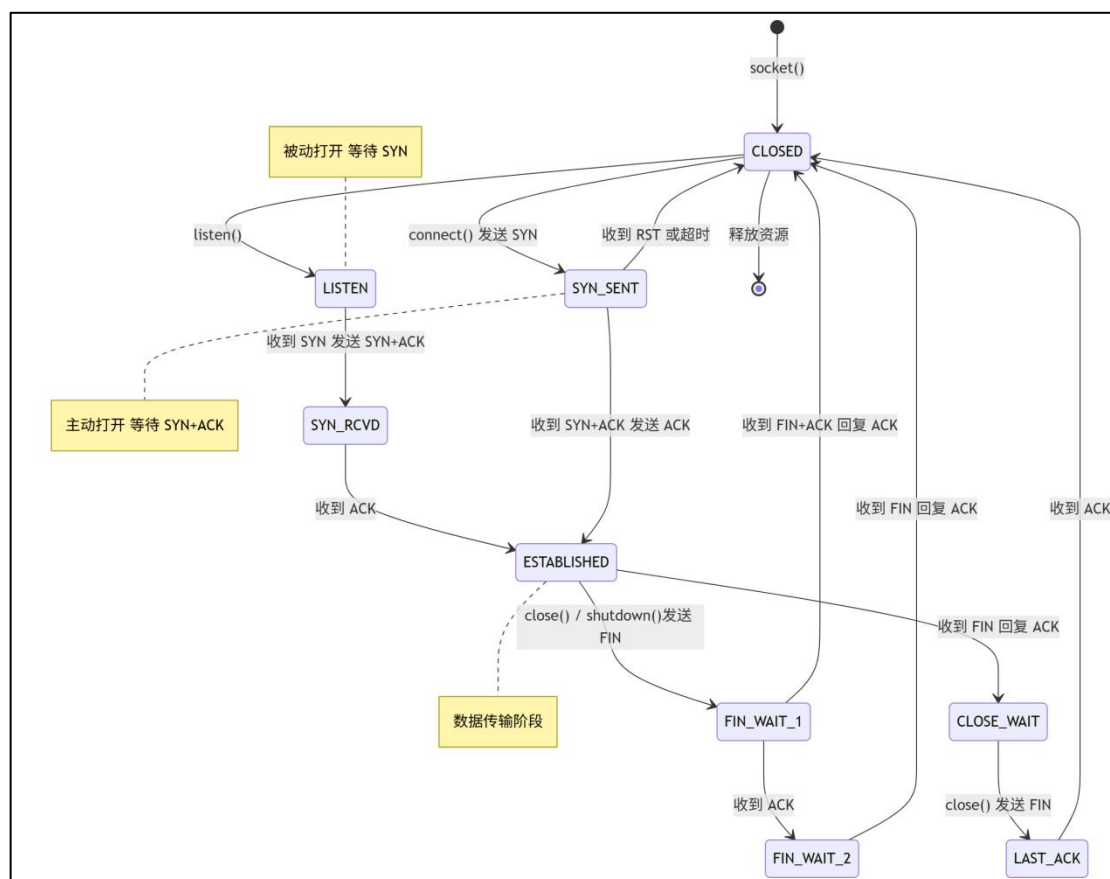


图8-2 TCP示意图

7.7 套接字层与文件系统集成

套接字层为用户态进程提供了标准的套接字 API，同时将套接字抽象为文件，无缝集成到Kairix的文件描述符管理系统中。核心数据结构Socket包含一个内部类型枚举SocketInner，可以是TCP、UDP、Raw或Unix套接字，此外还有文件描述符编号、所属进程ID、状态、关闭标志以及读写半关闭标志。

```
1 pub enum SocketInner {
2     Raw(Arc<Mutex<RawSocket>>),
3     Udp(Arc<Mutex<UdpSocket>>),
4     Tcp(Arc<Mutex<TcpSocket>>),
5     Unix(UnixSocket),
6 }
7 pub struct Socket {
8     pub inner: SocketInner,
9     pub fd: usize,
10    pub pid: usize,
11    pub state: SocketState,
12    pub closed: AtomicBool,
13    pub shut_rd: bool,
14    pub shut_wr: bool,
15    pub flags: u32,
16 }
```

为了使套接字能够像普通文件一样被读写，模块定义了SocketFile结构，它实现了File trait，其read和write方法根据套接字类型调用对应的传输层接收和发送函数，并支持非阻塞和阻塞模式：若操作无法立即完成，如接收队列为空或发送缓冲区满，则通过调用suspend_current_and_run_next让出当前协程，并将任务的唤醒器注册到套接字内部的recv_waker或send_waker中，待事件发生时唤醒任务继续执行。

此外，SocketFile还提供register_poll_waker和clear_poll_waker方法，支持epoll/select等I/O多路复用机制，将任务唤醒器与套接字事件关联。

```
1 fn register_poll_waker(&self, task: Arc<TaskControlBlock>) {
2     let pid = current_process().getpid();
3     let (tcp_socket, udp_socket) = {
4         let manager = SOCKET_MANAGER.lock();
5         let Some(sock) = manager.get_socket(self._fd, pid) else { return };
6         match &sock.inner {
7             SocketInner::Tcp(tcp) => (Some(tcp.clone()), None),
8             SocketInner::Udp(udp) => (None, Some(udp.clone())),
9             _ => (None, None),
10        }
11    };
12    if let Some(tcp) = tcp_socket {
13        let waker = task_waker_front(task);
14        let tcp_guard = tcp.lock();
15        if matches!(tcp_guard.state, TcpSocketState::Listening) {
16            *tcp_guard.accept_waker.lock() = Some(waker);
17        } else {
18            *tcp_guard.recv_waker.lock() = Some(waker);
19        }
20    }
```

```
19     }
20   } else if let Some(udp) = udp_socket {
21     udp.lock().set_waker(Some(task));
22   }
23 }
```

7.8 异步网络 I/O 模型

Kairix的网络栈深度集成了内核的异步协程调度器，所有可能阻塞的网络操作，如recvfrom、accept、connect和send，均被设计为可挂起的异步函数。当操作无法立即完成时，当前任务不会阻塞整个线程，而是通过调度器接口主动让出CPU，并将一个唤醒器Waker注册到相关套接字或连接对象上。该唤醒器封装了任务的标识和就绪状态，当底层事件发生时，相应的模块会调用唤醒器的wake方法，将任务重新放入调度器的就绪队列，从而在合适的时机恢复执行。

```
1 pub fn task_waker_front(task: Arc<TaskControlBlock>) -> Waker {
2     Waker::from(Arc::new(TaskWaker { task }))
3 }
4
5 struct TaskWaker { task: Arc<TaskControlBlock> }
6 impl Wake for TaskWaker {
7     fn wake(self: Arc<Self>) {
8         wakeup_task(self.task.clone());
9     }
10 }
11
12 pub fn wakeup_task(task: Arc<TaskControlBlock>) {
13     SCHEDULER.lock().add_task(task);
14 }
```

8 设备驱动

设备驱动是操作系统连接硬件和内核子系统的桥梁。Kairix当前主要面向QEMU virt平台，支持RISC-V64和LoongArch64两种架构。驱动层主要负责块设备、字符设备和网络设备的初始化与访问，并通过统一抽象向上层文件系统、网络协议栈和用户程序提供服务。

Kairix的设备驱动实现以VirtIO设备为核心：块设备主要服务于ext4根文件系统，字符设备通过devfs暴露给用户程序，网络设备则接入Kairix自研网络协议栈。我们队伍暂时还没有实现设备树，这是我们后续努力改进的地方。

8.1 设备解析

8.1.1 RISC_V MMIO 设备解析

在RISC-V64平台上，Kairix主要通过MMIO方式访问VirtIO设备。以VirtIO block为例，Kairix使用QEMU virt平台提供的VirtIO MMIO地址初始化块设备：

```
1 const VIRTIO0: usize = 0x10001000 + VIRT_ADDR_START;
2
3 let header = NonNull::new(VIRTIO0 as *mut VirtIOHeader).unwrap();
4 let transport = MmioTransport::new(header).unwrap();
```

在这种模式下，设备寄存器被映射到内核可访问的内存地址空间中，驱动通过读写这些MMIO寄存器与设备交互。Kairix在RISC-V64下将VirtIO block封装为：

```
1 pub struct VirtIOBlock(SleepLock<VirtIOBlk<VirtioHal, MmioTransport>>);
```

其中MmioTransport来自virtio-drivers，负责VirtIO MMIO传输层的具体交互。上层文件系统并不直接感知MMIO细节，而是通过统一的BlockDevice接口访问块设备。

8.1.2 LoongArch PCI 设备解析

在LoongArch64平台上，Kairix主要通过PCI方式发现和访问VirtIO设备。内核会从FDT中查找PCI host节点，并通过ECAM方式枚举PCI设备：

```
1 let pci_node = fdt.find_compatible(&["pci-host-ecam-generic"]).unwrap();
2 let cam = Cam::Ecaml;
3 let transport = super::pci::enumerate_pci(pci_node, cam).unwrap();
```

对于VirtIO block, LoongArch64下的设备封装为:

```
1 pub struct VirtIOBlock(SleepLock<VirtIOBlk<VirtioHal, PciTransport>>);
```

其中PciTransport负责PCI VirtIO设备的配置空间访问和传输层交互。相比RISC-V64的固定MMIO地址方式, PCI方式需要先枚举设备并找到对应的VirtIO transport, 再创建具体的块设备或网络设备实例。

Kairix的网络设备同样包含PCI探测逻辑。网络子系统启动时会尝试探测VirtIO-net设备并注册为eth0, 若探测失败, 则仍会保留loopback设备用于本地通信。

8.2 块设备驱动

块设备是文件系统的底层存储介质。Kairix当前主要使用VirtIO block作为磁盘设备, 并在其上挂载ext4根文件系统。块设备通过统一的BlockDevice抽象向文件系统提供读写接口。

Kairix使用全局BLOCK_DEVICE保存当前块设备实例:

```
1 lazy_static! {  
2     pub static ref BLOCK_DEVICE: Arc<dyn BlockDevice> =  
3         Arc::new(BlockDeviceImpl::new());  
4 }
```

VirtIO block 驱动封装在 VirtIOBlock 中。RISC-V64 下使用 MmioTransport, LoongArch64下使用PciTransport。

8.3 字符设备驱动

字符设备以字节流方式进行访问, 通常不支持随机寻址。Kairix中的字符设备主要通过devfs暴露给用户态程序, 挂载在/dev目录下。用户程序可以像访问普通文件一样访问这些设备。

对于底层控制台差异, 则由polyhal屏蔽, 使上层不需要关心具体架构的控制台

8.4 网络设备驱动

网络设备负责网络数据包的发送和接收。Kairix支持loopback设备和VirtIO-net设备，并通过统一的NetDevice trait接入网络协议栈。

```

1 pub trait NetDevice: Send + Sync {
2     fn name(&self) -> &str;
3     fn mtu(&self) -> u16;
4     fn flags(&self) -> NetDeviceFlags;
5     fn hard_start_xmit(&self, skb: Skb) -> Result<(Skb, u32, u16), &'static
    str>;
6     fn set_rx_handler(&self, handler: Box<dyn Fn(Skb) + Send + Sync>);
7     fn poll_rx(&self) {}
8     fn mac_addr(&self) -> [u8; 6] { [0; 6] }
9     fn ip_addr(&self) -> u32 { 0 }
10 }

```

其中，hard_start_xmit用于发送数据包，set_rx_handler用于注册接收回调，poll_rx用于轮询接收队列，mac_addr和ip_addr用于提供网络层所需的地址信息。

Kairix使用DeviceManager管理网络设备。网络子系统初始化时，内核会先注册loopback设备，并添加本地回环地址127.0.0.1。随后尝试探测VirtIO-net设备，如果探测成功，则注册为eth0，设置IP地址10.0.2.15，并添加默认路由；如果探测失败，则至少保留loopback设备，保证本地网络功能可用。

VirtIO-net收到数据包后，会通过RX handler将Skb交给以太网层：

```

1 virtio_net_arc.set_rx_handler(Box::new(move |mut skb| {
2     skb.dev = Some(rx_dev.clone());
3     if let Err(e) = ethernet_rcv(skb, rx_dev.clone()) {
4         log::info!("eth0 rx drop: {}", e);
5     }
6 }));

```

这样，设备驱动只负责数据包收发，而ARP、IPv4、ICMP、TCP、UDP等协议处理由上层网络栈完成。我们通过这种分层设计，将Kairix底层网络设备和协议栈解耦，使VirtIO-net和loopback可以复用同一套网络协议处理逻辑。

9 多架构设计

Kairix内核采用多架构模块化设计，目前通过移植和修改polyhal硬件抽象层实现了对RISC-V和LoongArch两种指令集架构的支持。

具体而言，Kairix利用Rust语言的条件编译机制，利用polyhal第三方库，实现硬件抽象层屏蔽了不同架构的底层差异。其核心理念是通过条件编译实现对应架构代码的自动选择，从而将不同架构硬件的功能抽象出来，供内核统一使用。

9.1 RISC-V和LoongArch指令集架构

9.1.1 RISC-V

RISC-V是由加州大学伯克利分校的研究团队于2010年推出的简单、可扩展的精简指令集架构，适用于从微控制器到高性能计算在内的广泛应用领域。

Kairix在启动阶段使用OpenSBI作为机器态固件，并通过SBI接口完成对底层硬件的访问和控制。Kairix通过将RISC-V的架构寄存器封装到统一框架中，使系统调用、缺页异常、非法指令、时钟中断等事件可以进入统一的内核处理流程。

9.1.2 LoongArch

LoongArch是由龙芯中科自主研发的精简指令集架构，具有较好的自主性、先进性、兼容性。与RISC-V相比，LoongArch在虚拟地址空间、特权寄存器、TLB 管理和异常返回机制上都有明显差异，因此Kairix对LoongArch提供了独立的底层适配。

9.2 Kairix硬件抽象层实现

硬件抽象层是操作系统内核与底层硬件之间的桥梁，用于屏蔽不同硬件平台的差异，提供统一的接口供上层内核使用。Kairix的硬件抽象层使用模块化设计，利用Rust语言的条件编译机制提供对内核底层模块的统一接口。

9.2.1 虚拟地址抽象

RISC-V和LoongArch在虚拟地址映射方面有显著差异。首先，RISC-V统一使用页表进行虚拟地址的映射；LoongArch提供更灵活的映射方式，允许内核分配几个DMW窗口实现对部分地址区间的直接映射，这段地址空间不通过页表进行翻译，Kairix主要使用DMW实现了对部分内核地址空间的直接映射。其次，RISC-V通过硬件填充TLB，仅需要软件在虚拟地址映射发生特殊变更时无效化TLB项；LoongArch则在无扩展的情况下不支持硬件填充TLB，要求软件全权管理TLB。

9.2.1.1 访存异常处理

在RISC-V中访存异常主要包括缺页异常和权限错误，用户程序访问尚未建立映射或权限不满足的虚拟地址时，硬件会直接触发页异常，在trap入口判断traptype并进入内存管理模块。若该地址属于合法VMA，内核会根据访问类型分配物理页、修正页表权限或完成COW复制，然后刷新对应TLB项。返回用户态后，用户程序重新访问该地址时，RISC-V MMU会根据新的页表项自动完成TLB填充，从而继续执行。

在LoongArch中。LoongArch的访存异常除缺页和权限错误外，还包括TLB重填异常。Kairix在trap初始化中设置了专门的tlb_fill入口，LoongArch用户态访问地址时，如果TLB中没有对应项，处理器首先进入TLB重填异常。如果页表项已经存在且有效，则直接执行tlbfill填入TLB，并返回用户态。如果页表中还没有合法映射，后续会转入普通页异常路径，由内核的统一缺页处理逻辑接管。

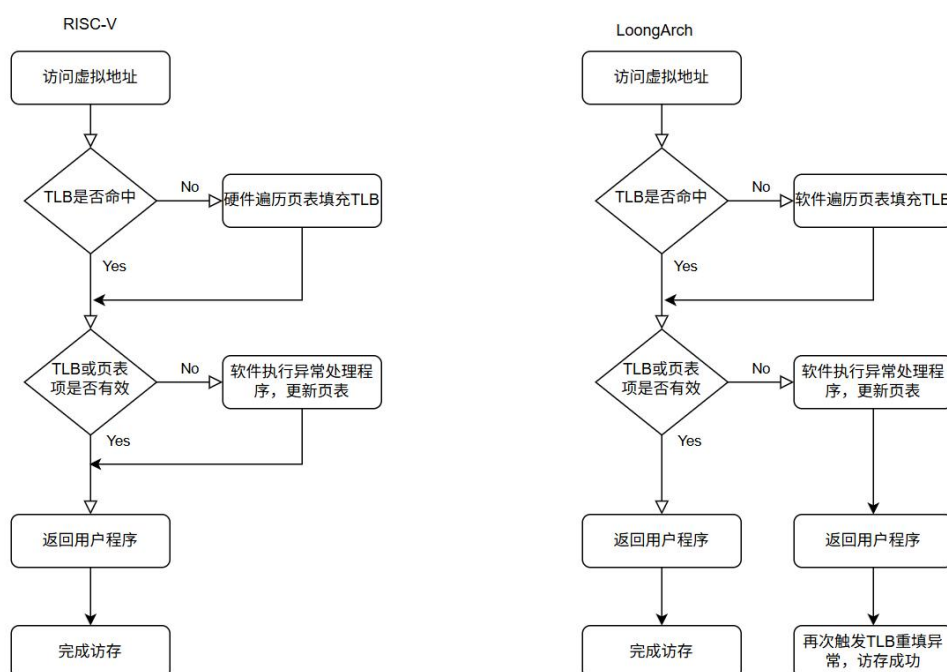


图2-1. RISC-V和LoongArch的异常处理路径

9.2.2 异常处理

异常处理是操作系统内核的核心功能之一。Kairix通过对异常上下文和异常类型进行封装，在RISC-V和 LoongArch两种架构上实现了统一的异常处理机制，使系统调用、时钟中断、非法指令、断点异常以及普通缺页异常等事件能够进入同一套上层软件处理逻辑。

9.2.2.1 架构差异和抽象

RISC-V和LoongArch的异常处理机制存在明显差异。两种架构不仅寄存器命名不同，异常原因寄存器、异常返回地址寄存器、中断控制寄存器和特权级状态寄存器的划分也不相同。例如，在RISC-V中，异常相关状态主要由sstatus、scause、stval、sepc、stvec 等CSR 表示；而在LoongArch中，类似功能被拆分到多个 CSR 中，异常原因由ESTAT表示，异常访问地址由BADV保存，异常返回地址为ERA，异常前特权状态由PRMD保存。

Kairix通过polyhal中TrapType的对这些差异进行抽象。底层架构代码负责读取各自的异常寄存器，并将其转换为统一的异常类型。

```

1    /// Enable interrupt
2    #[inline]
3    pub fn int_enable() {
4        crmd::set_ie(true);
5    }
6
7    /// Disable interrupt

```

```

8     #[inline]
9     pub fn int_disable() {
10         crmd::set_ie(false);
11     }

```

```

1    /// Enable interrupts.
2    #[inline]
3    pub fn int_enable() {
4        unsafe { set_sie() }
5    }
6
7    /// Disable interrupts.
8    #[inline]
9    pub fn int_disable() {
10        unsafe { clear_sie() }
11    }

```

9.2.2.2 异常处理上下文

RISC-V和LoongArch的异常类型会在读取各自的异常寄存器后被统一封装为以下类型：

```

1  #[derive(Debug, Clone, Copy)]
2  pub enum TrapType {
3      Breakpoint,
4      SysCall,
5      Timer,
6      Unknown,
7      SupervisorExternal,
8      StorePageFault(usize),
9      LoadPageFault(usize),
10     InstructionPageFault(usize),
11     IllegalInstruction(usize),
12     Irq(IRQVector),
13 }

```

RISC-V和LoongArch的采用了不同的异常处理上下文，都封装在TrapFrame中，代码如下：

```

1  #[repr(C)]
2  #[derive(Clone)]
3  // 上下文
4  pub struct TrapFrame {
5      pub x: [usize; 32], // 32 个通用寄存器
6      pub sstatus: Sstatus,
7      pub sepc: usize,
8      pub fsx: [usize; 2],
9  }

```

```

1  /// Saved registers when a trap (interrupt or exception) occurs.
2  #[allow(missing_docs)]
3  #[repr(C)]
4  #[derive(Debug, Default, Clone, Copy)]
5  pub struct TrapFrame {
6      /// General Registers

```

```
7     pub regs: [usize; 32],
8     /// Pre-exception Mode information
9     pub prmd: usize,
10    /// Exception Return Address
11    pub era: usize,
12 }
```

两种架构的上下文通过set、get方法或者TrapFrameArgs对外提供统一接口访问对应功能的寄存器:

```
1  /// Trap Frame Arg Type
2  ///
3  /// Using this by Index and IndexMut trait bound on TrapFrame
4  #[derive(Debug)]
5  pub enum TrapFrameArgs {
6      SEPC,
7      RA,
8      SP,
9      RET,
10     ARG0,
11     ARG1,
12     ARG2,
13     TLS,
14     SYSCALL,
15 }
```

10 AI工具的使用

大模型本质上是基于概率建模和大规模数据训练得到的工具。因此，在操作系统开发这种对正确性要求极高的工程中，不能把核心判断完全交给AI。所以在开发的过程中，我们团队利用了AI加快进度的开发，但是有限度的利用AI工具，我们认为使用AI的目的是：加快迭代的速度，减少重复工作量，而不是抛弃大脑。

我们使用的模型是kimi2.6+gpt5.5+deepseek。AI工具确实显著提高了我们的开发效率，但我们始终坚持有限度地使用AI：核心架构由人来设计，关键修改由人来审查，最终正确性由测试结果和代码阅读共同确认。

10.1 使用场景

Kairix的开发并不是完全的vibe coding，在初赛开发阶段，我们使用AI的场景主要有五个：

1. 辅助阅读代码。例如在阅读往届优秀代码、Linux的系统调用规范的时候，如果是直接阅读代码的话，容易因为局部的理解偏差导致整体结构的认知错误。因此我们团队会使用AI进行代码的初步阅读，给出我们一版凝练后的概述，让团队成员有个大致了解。再自顶向下，进行源码的阅读。在这个过程中，我们会带着批判性的思维去学习，在遇到模棱两可或者认为AI出错的地方，再和AI进行交流，进而加深代码的理解。

2. 重复性代码生成。因为AI工具本质上是概率学集大成体现，所以天生适合生成模式固定、重复性较强的工作，这在我们kairix的代码里面就有体现。例如在于非磁盘文件系统部分的编写时，涉及到不同实例的设计。例如/dev/zero、/dev/urandom、/dev/null 等文件对象的结构相似，初始化逻辑也高度重复。我们便采取人为写一版本示例，然后将重复性的工作交给AI。加快开发的效率的同时，也不容易出现问题。

```
1 ///
2 pub struct ZeroFile {
3     inner: Mutex<FileInner>,
4 }
5
6 impl ZeroFile {
7     ///
8     pub fn new(dentry: Arc<dyn Dentry>) -> Self {
9         Self {
10             inner: Mutex::new(FileInner {
11                 offset: 0,
12                 dentry,
13                 flags: OpenFlags::empty(),
14             }),}}}
```

```

1 pub struct UrandomFile {
2     inner: Mutex<FileInner>,
3 }
4
5 impl UrandomFile {
6     pub fn new(dentry: Arc<dyn Dentry>) -> Self {
7         Self {
8             inner: Mutex::new(FileInner {
9                 offset: 0,
10                dentry,
11                flags: OpenFlags::empty(),
12            }),
13        }
14    }
15 }

```

可以看到在这种重复性很强的工作里面，AI的完成度较高，可以减少机械重复工作。但涉及具体语义的部分，例如/dev/urandom如何生成随机数、/dev/zero的读写行为如何符合Linux语义，仍然需要人工检查和补充。

3. 辅助团队debug。我们团队会在代码关键节点中设置的日志。当出现死锁、panic等问题的时候，我们会将日志、具体的失败现象、以及我们个人的思考发给AI工具，利用AI给出初步的诊断。这个时候，我们会先看看初步的诊断符不符合我们所认为的问题出现的位置，然后再进行人为的调试确认。我们调试仅仅是让AI列出哪些地方可能出现问题，提升调试的效率，而不是完全让AI接管修复方案，因为AI经常给出看似合理，但完全不符合实际的判断。

4. 生成测试内容，一个好的代码，是离不开测试的。我们也会使用AI生成一些针对性测试程序，例如测试文件读写、管道阻塞与唤醒、信号投递、共享内存attach/detach、mmap权限等场景。AI生成测试的优势在于速度快，可以覆盖一些边界输入。但测试是否有效、是否符合Linux语义，仍然需要人工审查。

```

1 #include <stdio.h>
2 #include <string.h>
3 #include <unistd.h>
4 #include <sys/shm.h>
5 #include <sys/wait.h>
6
7 int main() {
8     int id = shmget(IPC_PRIVATE, 4096, IPC_CREAT | 0666);
9     char *p = shmat(id, 0, 0);
10
11     strcpy(p, "parent");
12
13     if (fork() == 0) {
14         strcpy(p, "child");
15         return 0;
16     }
17
18     wait(0);
19
20     if (strcmp(p, "child") == 0)

```

```
21     printf("shm ok\n");
22     else
23         printf("shm failed: %s\n", p);
24
25     shmdt(p);
26     shmctl(id, IPC_RMID, 0);
27     return 0;
28 }
```

这个测试核心就是：父进程映射共享内存，子进程写入，父进程检查是否可见。

5. 文档的润色，例如第十章的开头，我们就是用了AI的润色功能，原文是“大模型本质上是概率学和数据的应用，因此在写代码的时候，不能将全部的工作都放任AI工具的“碰运气”。”润色后是“大模型本质上是基于概率建模和大规模数据训练得到的工具。因此，在操作系统开发这种对正确性要求极高的工程中，不能把核心判断完全交给AI。”

因为AI的生成功能是根据上下文来的，因此在写文档的过程中，不能上来就让AI直接撰写文本，那样子出来的文章往往会偏离我们的想法。所以我们团队的文档撰写一般是，人工写一版初版，AI帮忙润色，然后我们再进行有选择的采纳。

10.2 AI生成内容范围

我们对AI生成内容进行了明确限制。我们会让AI辅助编写重复性的代码、文档润色，测试程序生成、日志分析、局部代码修改建议。

但是涉及到下面的内容，我们团队一定不会让AI参与决策：

1. 内核核心架构设计。
2. 调度、内存管理、信号、文件系统挂载等高风险模块的最终修改。
3. 涉及并发、锁顺序、生命周期和地址空间语义的关键逻辑。
4. 未经编译和测试验证的代码合入。
5. AI无法解释清楚原因的修改建议。

换句话说，我们团队认为AI可以参与“写”和“查”，但不能替代“判断”。

10.3 AI工具的幻觉

AI工具在开发中最大的风险是幻觉。它可能生成不存在的函数，引用错误的Linux语义，误判模块边界，或者把其他内核的设计强行套到Kairix上。尤其在操作系统开发中，一个看似合理的回答，可能会在锁、页表、信号返回、生命周期管理等细节上完全错误。

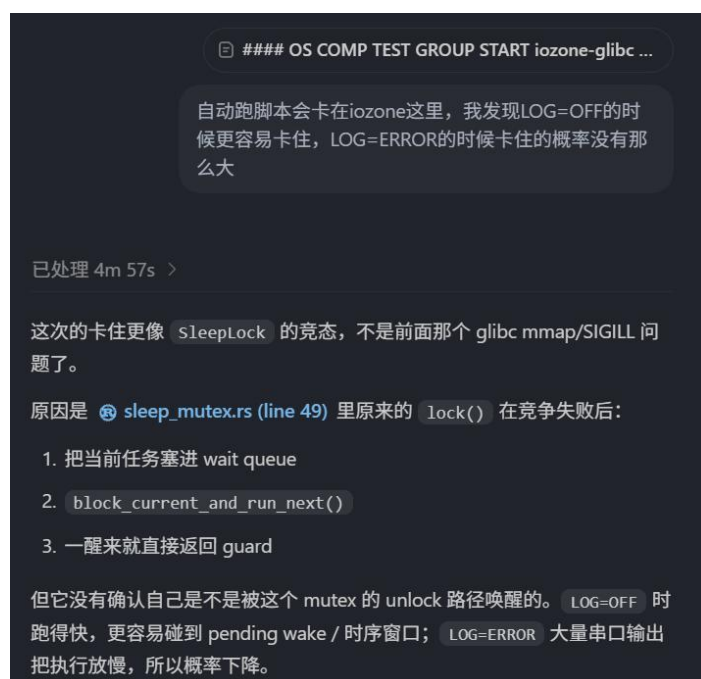


图10-1：AI出现幻觉的例子

这是一次我调试死锁问题的时候，ai刚开始是给出了glibc的mmap映射的问题，但是我认为问题不太可能出现在那个位置，因此我又重新跑了具体的日志，并且阐述问题发生的可能是因为LOG=OFF，也就是执行速度加快的时候所产生的竞态，这个时候ai才能大致的定位问题所在。

又比如，内核中我们有废弃的代码，已经不用了，但是AI分辨不出来这个，它会在已经废弃的语义上面进行修改，最终产生一个完全不符合当前我们内核的判断逻辑的代码。

我们认为，完全交给AI写代码，只会减少写代码的时间，但是真正debug的时候的时间就会大大变长，这是一个得不偿失的事情。因此，我们认为AI对Kairix的帮助更像是一个“加速器”，而不是“驾驶员”。它可以帮助我们更快地搜索、总结、生成和验证，但真正决定系统正确性的，仍然是我们团队对内核机制的理解、审查和测试。

10.4 反思

在六个多月的开发过程中，我们使用AI的方法也在不断迭代。在最初的时候，网页版大模型还是占据主导，我们将和网页版对话的习惯代入了与agent的交互，有疑惑的时候就提出prompt，这就导致不同的模块代码，不同的类型的prompt杂糅在一个上下文中，上下文越来越臃肿，指令之间相互干扰，早期的对话记录很快就被淹没在长串的滚动里。随着内核开发逐步深入，模块间的依赖关系日趋复杂，跨模块的约束条件成倍增长，这种方式的问题终于集中暴露出来：大量的约束被遗忘、内核模块的不清晰化、错误代码的扩散化。给最初的

团队开发带来了很大的阻碍，那个时候的我们，很多时候不是在往前推进功能，而是在回头修补AI无意间引入的连带问题。

到了中后期，这些踩过的坑慢慢被我们有意识地整理和规范。我们将内核的整体架构和设计写到AGENT.md里面，让agent在每次任务启动时都能加载这些前置知识，而不是依赖一次性的prompt传递。针对较大的实现，我们不再试图在一个回合里完成所有修改，而是拆分成多个职责单一、可独立验证的commit，方便团队进行二分法调试和回滚。这样规范化的流程，大大提升了我们团队的开发效率，同时也减少了AI使用带来的副作用，减少调试的时间。

并且和AI交互的这些时间，我们也逐渐形成了一个清晰的认知，并且随着时间推移愈发坚定：AI只是一个辅助，它不能代替人类进行思考，它是且仅是一个工具，而非大脑。它可以在给定的上下文里做快速的信息检索、模式匹配和代码生成，可以帮助我们重复性的、机械性的工作压缩到更短的时间内完成，但它并不理解系统为什么这样设计，也无法对未曾明确写入约束的边界做出可靠的判断。合理利用AI，可以让开发跑得更快，让团队把精力集中在真正需要判断力的地方；但最终的设计权衡、架构决策，始终全权掌握在我们团队自己手中。

11 总结与展望

11.1 主要工作成果

1. **测例通过方面**: 我们通过了除部分LTP测试之外的所有测试点, 在排行榜上位于前列

2. **文件系统与存储管理**: 接入并优化了lwext4文件系统, 在其官方Rust实现的基础上扩展了挂载等功能, 使其更符合实际应用需求。重构了虚拟文件系统(VFS), 使支持多种文件系统类型, 并实现了tmpfs、procfs、devfs等关键文件系统。引入页缓存机制, 显著提升了文件读写的性能。新增了与文件系统相关的系统调用, 完善了用户态对文件操作的访问能力。

3. **进程与信号机制**: 实现了标准的进程管理机制, 支持进程创建、销毁、调度等核心功能。按照Linux标准实现了信号处理机制, 支持标准信号和实时信号的正确传递与处理。通过内核空间动态映射, 优化了ELF文件加载, 支持动态链接, 提升了程序的加载效率。实现了futex机制和共享内存, 增强了进程间同步与通信能力。完善了IPC(进程间通信) 机制, 支持多种通信方式。

4. **内存管理**: 构建了支持分页、虚拟内存、缺页按需分配和写时复制的内存管理系统。Kairix通过VMA和页表权限共同管理用户地址空间, 在访问用户指针时进行地址合法性检查和权限校验, 并在必要时触发缺页处理。系统同时支持mmap、brk、用户栈扩展、共享内存和page cache等机制, 为用户程序提供较完整的虚拟内存支持。

5. **调度与多核支持**: Kairix具备基础的多核启动与调度支持, 能够在多核环境下初始化处理器并运行任务。调度器负责在可运行任务之间进行切换, 并结合阻塞、唤醒机制支持多个任务并发执行, 为后续进一步完善多核负载均衡和调度优化提供了基础。

6. **网络支持**: Kairix实现了自研网络协议栈, 支持loopback、VirtIO-net、ARP、IPv4、ICMP、TCP、UDP和raw socket等功能。通过VirtIO-net, Kairix能够在QEMU虚拟化环境中进行基本网络通信, 并支持ping、TCP/UDP连接、iperf、netperf等网络测试场景。

7. **硬件抽象与平台适配**: Kairix通过polyhal屏蔽不同架构的差异。同一份代码可以在不同硬件平台上运行, 体现了良好的可移植性和可维护性。

未来, 我们将继续优化系统性能, 增强稳定性和安全性, 以支持更广泛的应用场景。

11.2 开发过程

初赛阶段我们的内核搭建时间约为6个月，搭建过程主要分为以下4个阶段

1. **1月~2月：学习调研阶段。**队员首先系统性学习了rust语言语法，之后又学习了rCore-Tutorial并完成了对应的实验，最后调研了往届的优秀内核作品，为后续的内核搭建打下基础。
2. **2月~3月：进入基础开发阶段。**队伍在rCore的基础上进行内核开发，完成整体框架设计，并实现部分基础syscall。
3. **3月~4月：功能完善阶段。**陆续实现了懒分配、写时复制、VFS和页缓存，通过basic测例。同时学习LoongArch架构相关知识，并搭建网络栈。
4. **4月~6月：测试与适配阶段。**实现动态链接的支持，完善信号模块，支持BusyBox，并适配通过Lua、libc-test等测试用例，最终成功在龙芯架构下启动并调试运行。

11.3 开发经验总结

在项目开发过程中，我们积累了宝贵的经验：

1. **系统调用实现：**严格参考Linux System Calls Manual，确保与Linux系统调用的兼容性，实现了上百个系统调用，提升了系统兼容性和稳定性
2. **调试策略：**多核环境下的调试极具挑战性。死锁、阻塞唤醒等问题经常出现，且难以追踪。在debug的过程中，我们建立了完善的日志系统，通过运行时信息收集和分析来定位问题。对于glibc和musl相关测试，我们会结合libc实现和测试源码分析程序预期行为，再反推内核中可能出错的位置。
3. **代码质量：**在开发过程中，我们尽量保持不同模块之间边界清晰。对于复杂逻辑，我们会进行阶段性重构，减少重复代码和临时补丁。同时，我们会在在关键内部状态处使用断言或日志检查，尽早发现不符合预期的内核状态。
4. **性能优化：**因为性能优化需要依赖测试结果和实际瓶颈分析，因此我们在做性能优化时，主要是贴切着测试用例，进行通用性的优化。
5. **团队合作：**在开发的过程中，我们发现，一个提前规定好的团队git标准可以大大减少merge时候的工作量。所以我们在开发的过程中，会规定好什么时候merge，什么时候分开。同时我们团队保证一周至少一次的组会交流，共享代码设计思路。

11.4 遇到的问题 and 解决方案

11.4.1 网络回环设备收发问题

实现回环设备后，在进行数据包收发测试时，系统偶尔会卡死。回环设备的特殊之处在于，数据包进入底层发送路径后不会真正经过硬件设备，而是会被立即回送到网络栈上层。因此，如果发送路径中仍持有某些锁，回送过程中再次进入接收路径时就可能重新申请同一把锁，导致死锁。

为定位问题，我们在网络栈各层的关键路径中加入日志，包括发送入口、路由选择、回环设备发送函数、IP接收入口以及上层协议处理函数。通过日志确认卡死发生在回环包同步回灌后的锁申请位置。随后逐行检查相关代码，最终确认问题是由于回环设备在持锁状态下直接调用上层接收处理逻辑，导致锁未及时释放。

调整回环设备的处理逻辑，缩短锁的持有范围，避免在持有关键锁时直接回调上层协议栈。对于回环设备中的接收处理函数指针，使用RwLock保护，使注册handler和读取handler的并发访问更加安全；同时确保在调用接收处理逻辑前尽量释放不必要的锁，从而避免同步回灌路径中的死锁问题，提高网络栈稳定性。

11.4.2 旧设计产生的效率问题

在测试lmbench的分数的时候，我们团队发现，我们的每一项的分数都离官方的baseline相差甚远，再考虑到我们的iozone的分数也一直不是很理想。这个时候我就开始怀疑是不是一个通用路径产生的问题。而通用路径最基本最基本的就是syscall，于是我便将视角放在了syscall这里。

```
1  TrapType::SysCall => {
2      ctx.syscall_ok();
3      _set_sum_bit();
4      let args = ctx.args();
5      let syscall_id = ctx[TrapFrameArgs::SYSCALL];
6      let result = syscall(syscall_id, [
7          args[0], args[1], args[2], args[3], args[4], args[5],
8      ]);
9      match result {
10         Ok(val) => ctx[TrapFrameArgs::RET] = val,
11         Err(errno) => ctx[TrapFrameArgs::RET] = (-(errno.code() as isize)) as
12         usize,
13     }
14     TLB::flush_all();
15 }
```

我们团队在早期设计的时候，只追求功能的完善，没有考虑到效率。我们在每一次 `syscall` 过后，都采取了 `TLB::flush_all()`，这大大降低了内核在不需刷新页表的时候的系统调用的速度。这便是旧设计产生的问题，这种问题极难发现，因为没有日志，也不会让内核 panic。能够发现这个问题主要是 `lmbench` 的测试程序写的好。

因此，在后面开发的时候，我们团队经常考虑测试程序先行，通过测试程序，可以验证我们自己的设计是否合理，以及代码写的是否合理。

11.4.3 LoongArch架构的内存错误问题

刚开始跑 `ltp` 测例脚本的时候 LoongArch 架构下会在跑到上千个测例时候发生访存错误的问题导致内核意外退出。由于在单独跑出错的测例时不会出现问题，而且在 RISC-V 架构下跑测试脚本也没有出现问题，故分析问题的原因主要集中在任务过多时 LoongArch 内架构的存压力方面。

比较 LoongArch 和 RISC-V 的内存布局，发现二者的内核栈位置不同。此前 LoongArch 的所有内核空间使用 LoongArch 的 DMW 窗口进行直接映射，这使得内核栈地址也不经过页表翻译，而是通过 DMW 完成虚拟地址到物理地址的转换。该方式在系统启动和简单测试阶段可以正常工作，但在运行 LTP 等长时间、高压测试时，由于内核栈的虚拟地址有 `tid` 计算，在 `tid` 过大（如 2000+）的时候，会使用较大的虚拟地址空间并导致之前使用的物理地址无法复用，最终可能导致地址溢出。

为确定问题，我们首先更改内核栈的位置而不改变映射方式，发现在跑 `ltp` 脚本的时候依旧出错，但是出错的测例和正确运行的测例数量发生了变化，因此确定了问题确实出在内核栈地址上。故决定更改 LoongArch 内核栈地址空间和映射方式。我们首先更改了内核栈的地址空间，将内核栈放在内核地址高处，并更改上层对于内核栈地址的翻译方式，将内核栈的映射方式改为由页表翻译并由内存管理器分配物理页。在初步更改后 LoongArch 的内核栈并不能正常运行，因为 LoongArch 的页表寄存器包括 PGDL 和 PGDH，更改后的内核栈属于高地址空间，需要设置 PGDH 寄存器才能正常翻译，而先前的 LoongArch 页表切换只设置了 PGDL，之后我加上了对 PGDH 的设置，内核栈能够正常使用。

修改了这个问题之后 LoongArch 的 `ltp` 脚本基本已经可以运行完所以测例，同时也基本实现了 LoongArch 和 RISC-V 的基本统一，减少之后不同架构运行产生的结果差异。

11.5 项目意义

1. 技术成长：通过亲手实现操作系统的核心组件，我们更加深入地理解了内核的完整执行路径。开发过程中，我们也学会了在性能、正确性、兼容性和实现复杂度之间做工程权衡。同时也提高了我们Rust的水平。

2. 现实意义：Kairix验证了Rust在内核开发的可行性，为其他团队提供了参考。同时，项目中保留了较多设计文档、测试记录和关键代码注释，便于后续开发者继续学习、维护和扩展。

11.6 未来发展计划

- 1. 真实物理板的适配：**支持支持RISC-V以及龙芯开发板
- 2. 网络功能的完善：**支持更多的功能，比如http和ssh
- 3. 优化文件系统：**继续优化页缓存，同时补充负缓存等优化策略
- 4. 图形化界面：**提升用户交互体验
- 5. 锁的完善：**当前锁的种类还是不够，读写锁用的还是库里面的
- 6. 运行现实生活的应用：**例如数据库

12 参考往届内核

TitanixOS:no-std 锁

rCore:用户态程序

Polyhal、DelOnlx:多架构设计

PhoenixOS、Chronix、NighthawkOS: 设计文档

Chronix: 文件系统架构思路